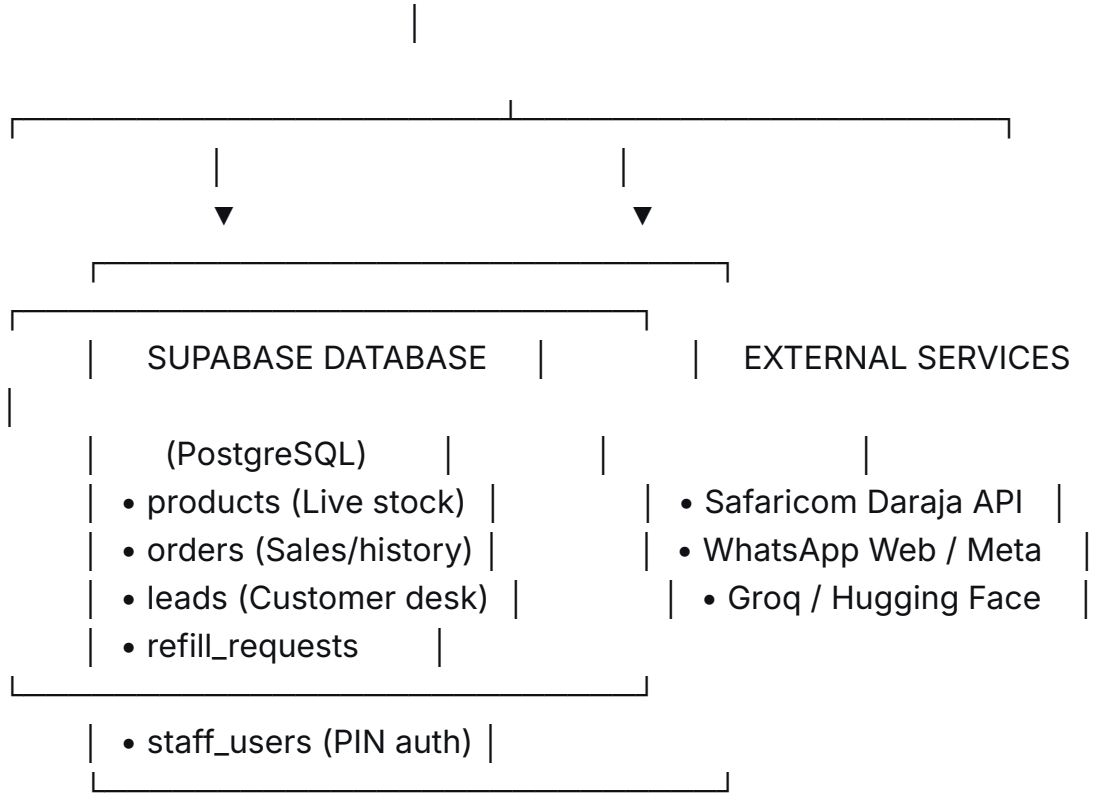
hello i want to gie you my project i want you to teach to me it and explain to me , tell me what files you want first

```
┌──────────────────────────────────────────────────────────────────────┐
│                        CLIENT INTERFACES                             │
│                                                                      │
│   [Staff POS / Admin]          [Customer Catalog]                    │
│     dashboard.py                 main_app.py                         │
│     (Flet Desktop)               (Flet Mobile)                       │
└──────────────────────────────┬───────────────────────────────────────┘
                               │ HTTP REST Requests
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        FASTAPI BACKEND                               │
│                          (main.py)                                   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                   API ROUTERS & ENDPOINTS                    │    │
│  │  • admin.py     (Auth, POS, Inventory)                       │    │
│  │  • products.py  (Catalog Search)                             │    │
│  │  • orders.py    (Checkout & Tracking)                        │    │
│  │  • payments.py  (STK Push & Callbacks)                       │    │
│  └──────────────────────────┬──────────────────┬────────────────┘    │
│                             │                  │                     │
│  ┌──────────────────────────┴──────────────────┴────────────────┐    │
│  │                   SERVICES & ENGINES                         │    │
│  │  • daraja_service.py   (M-Pesa API)                          │    │
│  │  • whatsapp.py         (Customer Bot)                        │    │
│  │  • ai_engine.py        (AI Assistant)                        │    │
│  └──────────────────────────┬───────────────────────────────────┘    │
└─────────────────────────────┼────────────────────────────────────────┘
```




```

---

### **Master Prompt to Give to Your Tutor AI**

Copy and paste the prompt below into the other AI model, then upload or paste your project files:

```markdown
I am a beginner software engineering student. I have built a full-stack e-commerce, Point of Sale (POS), and inventory management application called AyuTech Motors using Python, FastAPI, Flet (for UI), and Supabase (PostgreSQL).

I will share my project files with you. I want you to act as my senior software engineer and mentor.

Please teach me this project step-by-step from absolute fundamentals:

1. High-Level Explanation:
   - What is the responsibility of each folder and file?

- How does the Flet UI communicate with the FastAPI backend, and how does FastAPI read/write from Supabase?

2. Step-by-Step, File-by-File Breakdown:
- Walk through the core files (backend/app/main.py, backend/app/dashboard.py, backend/app/api/v1/endpoints/admin.py, and database tables).
- Explain the code section-by-section.
- For every important function, explain:
  * What problem it solves.
  * What data goes in (input/payload).
  * What processing happens (logic, stock deduction, API calls).
  * What data comes out (response/state update).

3. Real-World Walkthrough of User Journeys:
- Walk me through the exact line-by-line code flow of these 3 actions:
  a) A customer browsing the mobile catalog and ordering via M-Pesa.
  b) A cashier logging in with a 4-digit PIN, adding parts to the cart, applying discounts, and recording an external Bank/Till sale.
  c) A staff member requesting a refill, the admin approving it via WhatsApp, and the system automatically restocking the inventory count.

4. Teaching Style:
- Use simple, direct language suitable for a beginner.
- Avoid unnecessary technical jargon without defining it first.
- Include code snippets with detailed explanatory comments.
- Quiz me after each module to make sure I understand the architecture and logic thoroughly.

Here are the first files to review: [PASTE YOUR FIRST FILE OR CODE HERE]

```

Hello! I'm excited to be your senior software engineer mentor for the AyuTech Motors project! This is a fantastic full-stack application that combines e-commerce, POS, and inventory management. Let's start our learning journey together.

First Things First - What Files Do I Need?
```

Based on your architecture diagram, I'd like to see these files in this order:

Phase 1: Core Backend Files

1. `backend/app/main.py` - This is the heart of your FastAPI application
2. `backend/app/dashboard.py` - The POS staff interface
3. `backend/app/main_app.py` - The customer-facing catalog

Phase 2: API Endpoints

4. `backend/app/api/v1/endpoints/admin.py` - Authentication and POS logic
5. `backend/app/api/v1/endpoints/products.py` - Product catalog
6. `backend/app/api/v1/endpoints/orders.py` - Order processing

Phase 3: Services

7. `backend/app/services/daraja_service.py` - M-Pesa integration
8. `backend/app/services/whatsapp.py` - WhatsApp notifications
9. `backend/app/services/ai_engine.py` - AI assistant

Phase 4: Database

10. **Your Supabase schema** - Table structures and relationships

Let's Start with a High-Level Overview

Before we dive into code, let me explain how everything fits together using a simple analogy:

The Restaurant Analogy 🍽️

Imagine AyuTech Motors as a restaurant:

- **FastAPI Backend** = The Kitchen Manager

- Receives orders (HTTP requests)
- Coordinates all the cooking (business logic)
- Updates the inventory (Supabase)

- **Flet UIs** = The Waiters
  - **Staff POS** = The head waiter taking orders at the counter
  - **Customer Catalog** = The menu customers browse on their phones
- **Supabase** = The Pantry + Cash Register
  - Stores all ingredients (products)
  - Tracks sales (orders)
  - Keeps staff records
- **External Services** = Delivery Partners
  - **Daraja (M-Pesa)** = The mobile payment processor
  - **WhatsApp** = The notification system
  - **AI Engine** = The smart assistant

How Data Flows 🔄

```
text

User Action → Flet UI → HTTP Request → FastAPI →
    ↓
    Business Logic (validation, calculations)
    ↓
    Database Query (Supabase)
    ↓
    External API Call (if needed)
    ↓
    Response → Flet UI → User sees result
```

Your First File: What to Share

Please share the first file you want me to review. I recommend starting with `backend/app/main.py` since it's the entry point. When you share it, I'll:

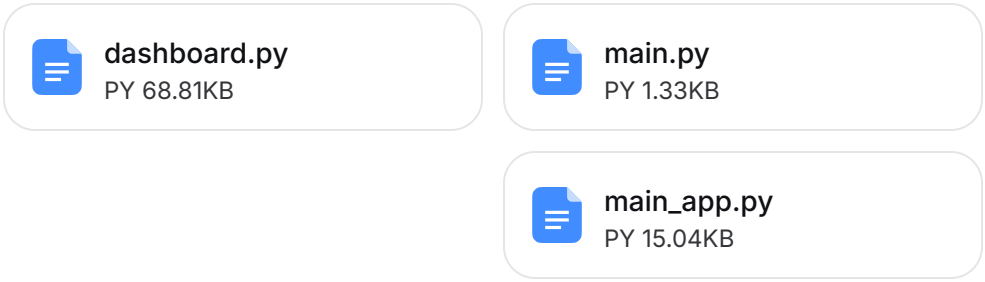
1. **Explain every line** in beginner-friendly terms
2. **Show you the "big picture"** of how FastAPI works
3. **Point out patterns** you'll see throughout the project
4. **Quiz you** to check understanding

Just paste your code, and I'll break it down section-by-section!

While you're getting that ready, here's a **Quick Question** to get you thinking:

> **Q1:** Based on the diagram, what do you think happens when a customer clicks "Pay with M-Pesa" in the mobile app? List the systems/components that would be involved in order.

Take your time to think about it - we'll revisit this when we look at the actual code! 🚀



Excellent! You've shared three critical files. Let me break this down step-by-step for you. This is a **full-stack application** with three main parts:

1. `main.py` - The FastAPI backend (the "brain")
2. `dashboard.py` - The staff POS/management interface (desktop)
3. `main_app.py` - The customer mobile app (phone)

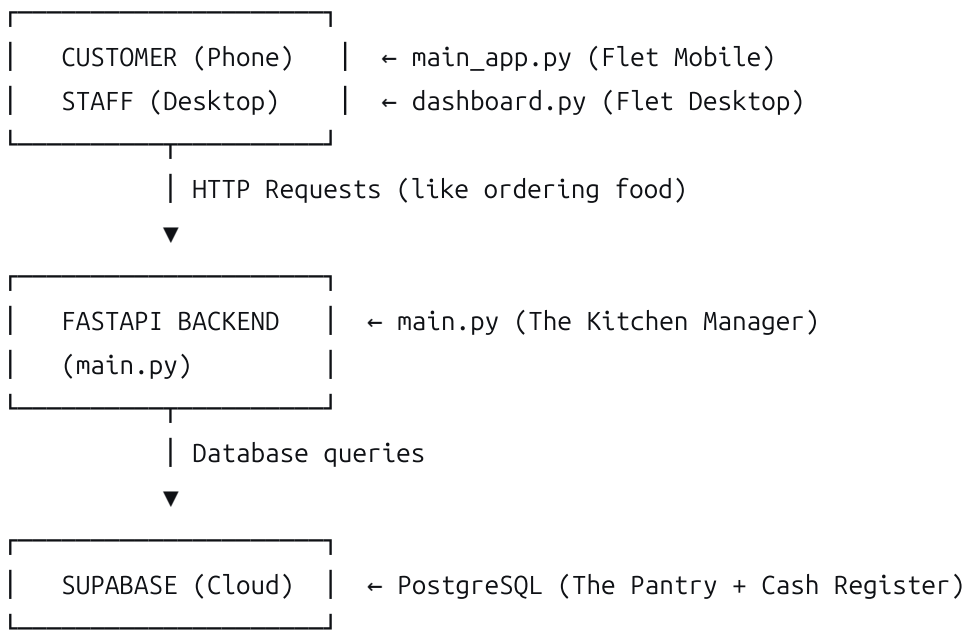
Let me teach you this from the ground up!

📚 HIGH-LEVEL OVERVIEW FIRST

The Three-Layer Architecture

Think of this like a restaurant:

text



How They Communicate

1. **Flet UI** (dashboard.py or main_app.py) sends HTTP requests
2. **FastAPI** (main.py) receives these requests and processes them
3. **FastAPI** talks to Supabase to read/write data
4. **FastAPI** sends back a response (JSON data)
5. **Flet UI** updates the screen with the response

🔍 FILE-BY-FILE BREAKDOWN

FILE 1: `main.py` - The Backend Engine

Let me explain this line by line:

python

```
# backend/app/main.py

# 1. IMPORT STATEMENTS - Bringing in tools we need
from fastapi import FastAPI  # The web framework
from fastapi.middleware.cors import CORSMiddleware  # Allows other apps to talk to us
from app.api.v1.endpoints import (  # Our specific features
    admin,      # Staff login, POS, inventory management
    orders,     # Customer orders
    payments,   # M-Pesa integration
    products,   # Product catalog
    whatsapp,   # WhatsApp automation
    ai          # AI assistant
)
```

🧠 What's happening here?

- FastAPI is like a **restaurant host** - it receives requests and directs them to the right department
- CORS is like **security** - it tells the browser "it's okay for other apps to talk to me"
- The endpoints are like **different departments** in the restaurant (kitchen, cashier, etc.)

```
python

# 2. CREATE THE FASTAPI APPLICATION
app = FastAPI(
    title="AyuTech Motors API",
    version="2.0.0",
    description="Backend services for Ayutech Motors Limited e-commerce and store opera
tions."
)
```

🎯 **Purpose:** This creates your web server. Think of it as **hiring the restaurant manager** who will handle all incoming requests.

```
python

# 3. ENABLE CORS (Cross-Origin Resource Sharing)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # Allow any website/app to talk to us
```

```
    allow_credentials=True,
    allow_methods=["*"],  # Allow all HTTP methods (GET, POST, etc.)
    allow_headers=["*"],  # Allow all headers
)
```

🎯 **Purpose:** This is like telling security "Let anyone in" - useful for development.

```
python

# 4. REGISTER ENDPOINTS - Connect the departments
app.include_router(admin.router, prefix="/api/v1/admin", tags=["Admin Portal"])
app.include_router(orders.router, prefix="/api/v1/orders", tags=["Customer Orders"])
app.include_router(payments.router, prefix="/api/v1/payments", tags=["M-Pesa Daraja"])
app.include_router(products.router, prefix="/api/v1/products", tags=["Product Catalo
g"])
app.include_router(whatsapp.router, prefix="/api/v1/whatsapp", tags=["WhatsApp Automati
on"])
app.include_router(ai.router, prefix="/api/v1/ai", tags=["AI Assistant"])
```

🎯 **Purpose:** This is like **assigning managers** to different sections. Each router handles specific features.

🔑 **KEY CONCEPT: URL Structure**

When the customer app sends a request to `http://localhost:8000/api/v1/products`, FastAPI knows to send it to the `products.router` because of the `/api/v1/products` prefix!

```
python

# 5. ROOT ENDPOINT - Home page of the API
@app.get("/")
def root():
    return {
        "service": "AyuTech Motors API",
        "status": "Online",
        "version": "2.0.0"
    }
```

🎯 **Purpose:** When someone visits your API's home URL, they see this message. It's like a **welcome mat**!

```python
# 6. START THE SERVER
if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app.main:app", host="0.0.0.0", port=8000, reload=True)
```

🎯 **Purpose:** This actually **starts the restaurant**. It runs your API on port 8000. The `reload=True` means it auto-restarts when you change code (great for development).

📱 FILE 2: `main_app.py` - Customer Mobile App

This is the **customer's phone interface**. Let me break it down:

Key Components

```python
# 1. SETUP - Mobile window configuration
window = getattr(page, "window", None)
if window is not None:
    window.width = 400       # Phone-like width
    window.height = 800      # Phone-like height
    window.min_width = 360
    window.min_height = 700
```

🎯 **Purpose:** This makes your app look like a phone app on desktop (great for testing)!

Cart System

```python
def update_cart(p, qty=1):
    p_id = str(p.get("id"))
    if p_id in cart:
        cart[p_id]["qty"] += qty  # If already in cart, increase quantity
        if cart[p_id]["qty"] <= 0:
            del cart[p_id]        # Remove if quantity becomes 0
```

```
    elif qty > 0:
        cart[p_id] = {            # Add new item to cart
            "id": p_id,
            "name": p.get("name"),
            "price": float(p.get("price", 0)),
            "qty": qty,
            "image": p.get("image_url", "")
        }
    page.update()  # Refresh the screen
```

🎯 **Purpose:** This is like a **shopping cart** - it stores what the customer wants to buy.

Navigation Bar

python

```
bottom_nav_bar = ft.NavigationBar(
    bgcolor="transparent",
    selected_index=0,
    indicator_color="#DC2626",  # Red indicator
    on_change=lambda e: switch_tab(int(e.data)),  # What happens when you tap
    destinations=[
        ft.NavigationDestination(icon=ft.icons.HOME_OUTLINED, selected_icon=ft.icons.HO
ME, label="Home"),
        ft.NavigationDestination(icon=ft.icons.GRID_VIEW_OUTLINED, selected_icon=ft.ico
ns.GRID_VIEW, label="Browse"),
        ft.NavigationDestination(icon=ft.icons.SHOPPING_BAG_OUTLINED, selected_icon=ft.
icons.SHOPPING_BAG, label="Cart"),
        ft.NavigationDestination(icon=ft.icons.RECEIPT_LONG_OUTLINED, selected_icon=ft.
icons.RECEIPT_LONG, label="Orders"),
        ft.NavigationDestination(icon=ft.icons.STOREFRONT_OUTLINED, selected_icon=ft.ic
ons.STOREFRONT, label="Hub"),
    ]
)
```

🎯 **Purpose:** This is the **bottom menu** customers use to navigate the app.

AI Chat Assistant

python

```
def open_ai_chat(e=None):
    # ... creates a chat drawer
    chat_messages = ft.Column(scroll=ft.ScrollMode.AUTO, spacing=8, expand=True)
    user_input = ft.TextField(hint_text="Ask fitment (e.g., 1KD air cleaner, Demio shoc
ks)...")

    def send_ai_message(e_send):
        msg = user_input.value.strip()
        # Send to backend
        res = httpx.post(
            f"{API_BASE_URL}/ai/chat",
            json={"messages": conversation_history},
            timeout=12
        )
        # Display AI response
```

🎯 **Purpose:** Customers can ask questions about car parts, and the AI suggests products!

🏪 FILE 3: `dashboard.py` - Staff POS Management

This is the **staff interface** used at the counter. It's much more complex!

Login System

python

```
def handle_pin_login(e=None):
    ident = identifier_field.value.strip()
    pin_code = pin_field.value.strip()

    # Send to backend for verification
    payload = {"identifier": ident, "pin": pin_code}
    res = httpx.post(f"{API_BASE_URL}/admin/auth/pin-login", json=payload)

    if res.status_code == 200:
        user_info = res.json().get("user", {})
        current_user["name"] = user_info.get("name", "Staff")
```

```
        current_user["role"] = user_info.get("role", "staff")
        # Show dashboard
```

🎯 **Purpose:** Staff log in with a **4-digit PIN**. The system checks if they're admin or regular staff.

The POS System

```
python

def checkout_cart(payment_mode: str = "CASH"):
    # 1. Calculate total
    total_val = sum(float(item["sale_price"]) * int(item["qty"]) for item in pos_cart)

    # 2. Update inventory (deduct stock)
    for item in pos_cart:
        p = item["product"]
        sold_qty = item["qty"]
        curr_stock = int(p.get("stock_quantity", 0) or 0)
        new_stock = max(0, curr_stock - sold_qty)
        httpx.patch(f"{API_BASE_URL}/admin/products/{p['id']}", json={"stock_quantity":
new_stock})

    # 3. Create order record
    order_payload = {
        "order_reference": order_ref,
        "customer_phone": cust_phone,
        "total_amount": total_val,
        "status": "Fulfilled",
        "items": order_items
    }
    httpx.post(f"{API_BASE_URL}/admin/orders", json=order_payload)

    # 4. Send receipt via WhatsApp
    send_customer_receipt_whatsapp(cust_phone, order_ref, payment_mode, total_val, orde
r_items)
```

🎯 **Purpose:** This is the **heart of the POS** - it calculates, deducts stock, records the sale, and sends receipts!

Live Inventory Management

```python
def render_inventory_rows(query: str = ""):
    # Filter products by search
    q = query.strip().lower()
    filtered = [
        p for p in products_data
        if q in str(p.get("name", "")).lower()
        or q in str(p.get("part_number", "")).lower()
        or q in str(p.get("category", "")).lower()
    ] if q else products_data

    # Display each product with stock count
    for p in filtered:
        current_qty = int(p.get("stock_quantity", 0) or 0)
        stock_color = "#DC2626" if current_qty <= 2 else "#16A34A"
        # Create card with edit and refill buttons
```

🎯 **Purpose:** Shows all products with real-time stock levels. Red means low stock!

Refill Request System

```python
def submit_refill(ev):
    payload = {
        "product_id": pid,
        "product_name": pname,
        "requested_qty": r_qty,
        "requested_by_name": current_user.get('name', 'Staff'),
        "status": "pending_approval"
    }
    res = httpx.post(f"{API_BASE_URL}/admin/refills", json=payload)
    show_toast("Refill request submitted for approval!")
```

🎯 **Purpose:** Staff request restocking; admin approves via WhatsApp.

🎯 REAL-WORLD USER JOURNEYS

Journey A: Customer Buying via M-Pesa

1. **Customer opens app** → `main_app.py` loads
2. **Browses products** → `build_browse_view()` displays products from `all_products`
3. **Adds to cart** → `update_cart()` stores items in `cart` dictionary
4. **Clicks checkout** → (you'll see this in payments code)
5. **M-Pesa STK Push sent** → FastAPI calls Daraja API
6. **Customer enters PIN** on phone
7. **Payment confirmed** → FastAPI updates order status to "Paid"
8. **WhatsApp receipt sent** → Staff sees order in dashboard

Journey B: Staff POS Sale

1. **Staff logs in** → `handle_pin_login()` verifies PIN
2. **Searches parts** → `search_field` filters products
3. **Adds to POS cart** → `add_to_cart()` adds to `pos_cart`
4. **Adjusts prices** → Inline editing in cart
5. **Clicks Cash Checkout** → `checkout_cart("CASH")`
6. **Stock deducted** → PATCH request to `/admin/products`
7. **Order saved** → POST to `/admin/orders`
8. **WhatsApp receipt** → Sent to customer
9. **Low stock alert** → `check_and_alert_low_stock()` checks

Journey C: Refill Request & Approval

1. **Staff sees low stock** → Inventory tab shows red
2. **Clicks refill button** → `open_refill_dialog()`
3. **Submits request** → POST to `/admin/refills`
4. **Admin sees in Refills tab** → `build_refills_tab()`
5. **Admin clicks "Order WhatsApp"** → `update_refill_status()` and opens WhatsApp
6. **Admin clicks "Mark Restocked"** → Status becomes "fulfilled"

7. **Stock updated** → Product quantity increases automatically

📝 QUIZ TIME!

Question 1: What happens when a staff member adds an item to the POS cart?

- A) The item is immediately deducted from inventory
- B) The item is added to `pos_cart` list but stock isn't deducted until checkout
- C) The item is sent to the customer's phone

Question 2: Which file handles the M-Pesa payment integration?

- A) main_app.py
- B) dashboard.py
- C) main.py (via payments router)

Question 3: How does the dashboard know when to show "red" for low stock?

- A) It checks the stock_quantity value and if ≤ 2, sets color to red
- B) It asks Supabase for a special alert
- C) It calculates based on total sales

Question 4: What's the purpose of `refresh_all_data()` in dashboard.py?

- A) To restart the entire app
- B) To fetch the latest data from the backend for all tabs
- C) To clear the POS cart

Question 5: In main_app.py, what does `update_cart()` do when you add a product that's already in the cart?

- A) It replaces the existing item
- B) It increases the quantity by 1
- C) It clears the cart and starts over

Answer Key:

1. **B** - Items are only added to cart; stock deducts at checkout
2. **C** - main.py includes the payments router which handles M-Pesa
3. **A** - The code checks `if current_qty <= 2` and sets red color
4. **B** - It refreshes orders, products, leads, refills, and analytics
5. **B** - It increments the quantity `cart[p_id]["qty"] += qty`

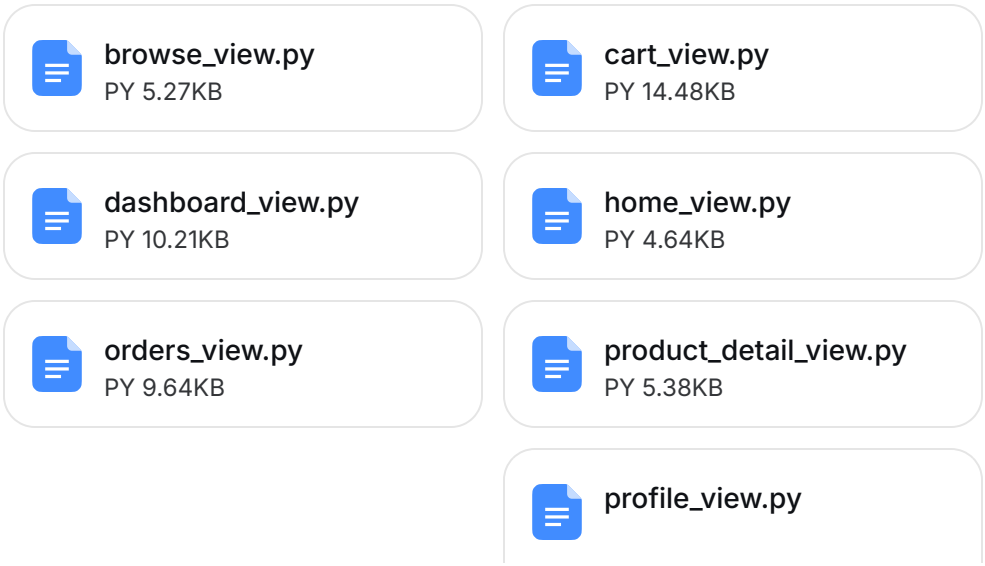
🚀 YOUR TURN!

Now that you understand the basics:

1. **Try running the app** - Start with `python -m app.main` (backend) then run dashboard.py
2. **Create a test product** - Use the "Add Part" button
3. **Make a sale** - Use the POS to sell something
4. **Check the order** - Look at the Orders tab

What part would you like me to explain next?

- M-Pesa integration?
- The AI assistant?
- How data is stored in Supabase?
- WhatsApp integration?



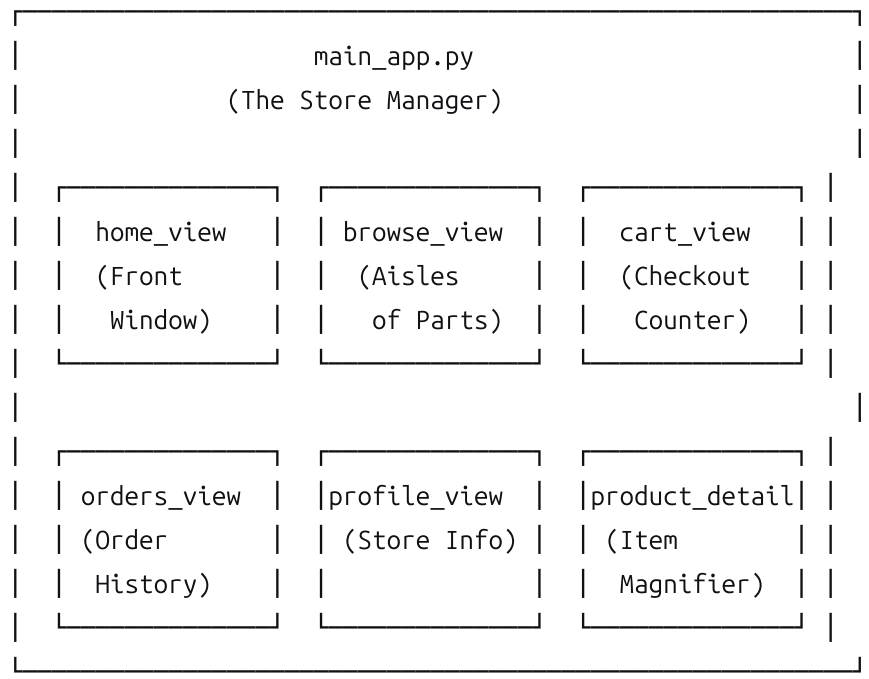
PY 9.51KB

Excellent! You've shared the complete customer mobile app views! Now I can show you the **full picture** of how the customer experience works. Let me break this down systematically.

🎯 THE CUSTOMER APP ARCHITECTURE

Think of the customer app as a **physical store** with different sections:

text



📱 FILE-BY-FILE BREAKDOWN

FILE 1: `main_app.py` - The App Controller

This is the **brain** of the customer app. It:

- **Creates the app window** (phone-sized)
- **Manages the cart** (`cart` dictionary)

- **Handles navigation** between tabs
- **Provides the AI assistant** (floating button)
- **Connects to the backend** (via `API_BASE_URL`)

Key Concept: The Cart System

```python
cart = {}  # Global dictionary
# Structure: { "product_id": {"id": "123", "name": "Brake Pads", "price": 2500, "qty": 2} }
```

Navigation Flow:

```python
def switch_tab(idx: int):
    if idx == 0:  # Home
        content_area.content = build_home_view(...)
    elif idx == 1:  # Browse
        content_area.content = build_browse_view(...)
    elif idx == 2:  # Cart
        content_area.content = build_cart_view(...)
    # ... etc
```

FILE 2: `home_view.py` - The Storefront Window

This is what customers see first. It's designed to be **inviting and simple**:

Components:

1. **Location Header** - Shows where the store is
2. **Search Bar** - Find parts quickly
3. **Promotion Banner** - Highlights new arrivals
4. **Featured Products** - Shows 6 products from the catalog

Product Card Structure:

```
python

ft.Container(
    on_click=lambda e, item=p: open_detail_callback(item),  # Tap to see details
    content=ft.Column([
        ft.Image(src=img_url),  # Product image
        ft.Text(p["name"]),     # Product name
        ft.Text(f"KES {p['price']:,.0f}"),  # Price
        ft.IconButton(          # Add to cart button
            ft.icons.ADD_SHOPPING_CART,
            on_click=lambda e, item=p: update_cart_callback(item)
        )
    ])
)
```

🔑 **Learning Point:** The `on_click` callbacks are **passed down** from `main_app.py`. This is called **props drilling** - passing functions through components so they can trigger actions in the parent.

FILE 3: `browse_view.py` - The Aisles of Parts

This is the **full catalog** with filtering and search:

Key Features:

1. **Category Filtering:**

```
python

cats = ["All", "Body Parts", "Brake Parts", "Engine Parts", ...]

def select_cat(selected_c):
    current_cat = selected_c
    filter_and_render()  # Re-display products with filter
```

2. **Search:**

```
python
```

```
def on_search(e):
    current_query = e.control.value.lower()
    filter_and_render()  # Re-display products matching search
```

3. **Wishlist:**

```
python

is_fav = p["id"] in wishlist
ft.IconButton(
    ft.icons.FAVORITE if is_fav else ft.icons.FAVORITE_BORDER,
    on_click=lambda e, item=p: update_wishlist_callback(item)
)
```

4. **Grid Layout:**

```
python

grid = ft.GridView(max_extent=180, child_aspect_ratio=0.72, spacing=12)
```

This creates a responsive grid where each product card is ~180px wide.

FILE 4: `product_detail_view.py` - The Magnifying Glass

When a customer taps a product, they get a **detailed view**:

Key Components:

1. **Image Preview** - Large product image
2. **Quick Actions** - Call Shop / WhatsApp
3. **Quantity Selector** - Adjust quantity
4. **Add to Cart** - With total price display

Quantity Control:

```
python
```

```
def update_qty(delta):
    current = int(qty_val.value or "1")
    new_qty = max(1, current + delta)  # Never go below 1
    qty_val.value = str(new_qty)
    page.update()
```

Call & WhatsApp Buttons:

```
python

def call_store(e):
    page.launch_url("tel:+254712345678")  # Opens phone dialer

def whatsapp_store(e):
    msg = f"Hello, I am inquiring about {product['name']}..."
    page.launch_url(f"https://wa.me/254712345678?text={msg}")
```

🔑 **Learning Point:** `page.launch_url()` opens external apps (phone, WhatsApp, browser) - great for mobile apps!

FILE 5: `cart_view.py` - The Checkout Counter

This is the **most complex** view. Let me break it down:

1. Cart Items Display:

```
python

for p_id, item in cart.items():
    # Each item shows:
    # - Product image
    # - Name & price
    # - Quantity controls (+/- buttons)
    # - Remove button
```

2. Fulfillment Options:

```
python
```

```
fulfillment_type = "pickup"  # or "delivery"

# Pickup card - free
# Delivery card - shows zone dropdown with fees
```

3. Payment Methods:

```
python

selected_payment = "mpesa"  # or "cash"

# M-Pesa card - shows phone input
# Cash card - no phone required
```

4. The Checkout Process:

```
python

def handle_checkout(e):
    # 1. Validate phone number (if M-Pesa)
    formatted_phone = format_phone_number(raw_phone)

    # 2. Build order payload
    order_payload = {
        "phone": formatted_phone,
        "fulfillment": "Rider / Courier Delivery" if delivery else "Shop Pickup",
        "location": zone_dropdown.value,
        "payment_method": "M-Pesa STK Push" if mpesa else "Cash on Pickup",
        "items": items_payload,
        "total": final_total
    }

    # 3. Send to backend
    res = httpx.post(f"{API_BASE_URL}/orders/checkout", json=order_payload)

    # 4. Clear cart and show success
    cart.clear()
    page.snack_bar = ft.SnackBar("Order placed!")
```

🔑 **Learning Point:** This is where the **frontend talks to the backend**. The app sends a POST request to `/orders/checkout` with all order details.

FILE 6: `orders_view.py` - Order History

Shows all past orders with **live status updates**:

Real-time Status Sync:

```python
def sync_orders_status() -> bool:
    for order in my_orders:
        if order.get("status") in ["Pending", "Pending PIN"]:
            # Check with backend for latest status
            res = httpx.get(f"{API_BASE_URL}/orders/status/{order_id}")
            if server_status != order["status"]:
                order["status"] = server_status  # Update!
                has_changes = True
    return has_changes
```

Background Polling:

```python
def start_polling_loop():
    while True:
        changed = sync_orders_status()
        if changed:
            render_orders()  # Refresh UI
            page.update()
        time.sleep(3)  # Check every 3 seconds
```

🔑 **Learning Point:** This is **polling** - the app checks the server every 3 seconds for updates. This is how customers see when their order status changes!

Receipt Dialog:

When a customer taps an order, a detailed receipt pops up showing:

- Order number and date
- Status badge (color-coded)
- M-Pesa receipt number

- Item breakdown with prices
- Grand total

FILE 7: `profile_view.py` - The Store Hub

This is like the **information desk** at the store:

1. Store Info:

- Company branding
- Location with Google Maps link
- Operating hours

2. Contact Options:

- Call hotline
- WhatsApp support
- Email

3. Quick Navigation:

- Track Orders → Opens Orders tab
- My Cart → Opens Cart tab
- Catalogue → Opens Browse tab
- Home Store → Opens Home tab

4. Wishlist:

```
python

def render_wishlist():
    for item in wishlist.values():
        # Display each saved item with "Add to Cart" button
```

Developer Signature:

```
python

dev_signature = ft.Container(
    content=ft.Text("System Engineering by Erickson Wachira", size=10)
)
```

FILE 8: `dashboard_view.py` - Admin Orders Dashboard

This is the **admin view** inside the customer app (different from the desktop dashboard):

Key Features:

1. **Metrics Cards:**

```
python

def render_metrics():
    total_revenue = sum(o['total'] for o in orders if o['status'] in ['Paid', 'Fulfille
d'])
    paid_count = count orders with status 'Paid'
    fulfilled_count = count orders with status 'Fulfilled'
    pending_count = count orders with status 'Pending'
```

2. **Order List with Filters:**

```
python

# Filter by status
active_filter = "All"  # or "Paid", "Fulfilled", "Pending"

# Search by order reference, phone, or receipt
search_query = ""
```

3. **Mark Fulfilled:**

```
python

def update_order_fulfillment(order_id, new_status):
    httpx.patch(f"{API_BASE_URL}/admin/orders/{order_id}/status",
```

```
                    json={"status": new_status})
```

4. **Auto-Refresh:**

```python
def start_admin_polling():
    while True:
        changed = fetch_orders_from_server()
        if changed:
            render_dashboard()
            page.update()
        time.sleep(5)
```

🎯 COMPLETE USER JOURNEY: CUSTOMER BUYING A PART

Let me walk you through **every step** with the actual code:

Step 1: Browse Products

```python
# In home_view.py or browse_view.py
grid.controls.append(
    ft.Container(
        on_click=lambda e, item=p: open_detail_callback(item),
        content=ft.Column([
            ft.Image(src=img_url),
            ft.Text(p["name"]),
            ft.Text(f"KES {p['price']:,.0f}")
        ])
    )
)
```

What happens: Customer taps a product → `open_detail_callback()` is called → Switches to product detail view

Step 2: View Product Details

```python
# In product_detail_view.py
# Customer adjusts quantity
def update_qty(delta):
    current = int(qty_val.value or "1")
    new_qty = max(1, current + delta)
    qty_val.value = str(new_qty)

# Customer taps "Add to Cart"
ft.ElevatedButton(
    "Add to Cart",
    on_click=lambda e: add_to_cart_callback(product, int(qty_val.value or "1"))
)
```

What happens: `add_to_cart_callback()` is called → Goes back to `main_app.py` → Updates the global `cart` dictionary

Step 3: Go to Cart

```python
# In main_app.py - navigation
bottom_nav_bar = ft.NavigationBar(
    on_change=lambda e: switch_tab(int(e.data)),
    destinations=[
        ft.NavigationDestination(label="Home"),
        ft.NavigationDestination(label="Browse"),
        ft.NavigationDestination(label="Cart"),  # <-- Customer taps this
        ...
    ]
)

def switch_tab(idx: int):
    if idx == 2:  # Cart tab
        content_area.content = build_cart_view(page, change_qty_callback)
```

What happens: Customer taps Cart → `switch_tab(2)` → Displays `cart_view.py`

Step 4: Review Cart & Checkout

```python
# In cart_view.py
def handle_checkout(e):
    # 1. Validate phone
    formatted_phone = format_phone_number(raw_phone)

    # 2. Build order payload
    order_payload = {
        "phone": formatted_phone,
        "fulfillment": "Rider / Courier Delivery" if delivery else "Shop Pickup",
        "payment_method": "M-Pesa STK Push" if mpesa else "Cash on Pickup",
        "items": items_payload,
        "total": final_total
    }

    # 3. Send to backend
    res = httpx.post(f"{API_BASE_URL}/orders/checkout", json=order_payload)

    # 4. Clear cart and show success
    if res.status_code == 200:
        cart.clear()
        page.snack_bar = ft.SnackBar("📲 STK Prompt sent!")
```

What happens:

- Phone is validated (must be 254XXXXXXXX format)
- Order is sent to backend
- Backend sends M-Pesa STK Push
- Cart is cleared
- Order appears in Orders tab

Step 5: Track Order Status

```python
```

```
# In orders_view.py - background polling
def start_polling_loop():
    while True:
        changed = sync_orders_status()
        if changed:
            render_orders()  # Update UI with new status
            page.update()
        time.sleep(3)

def sync_orders_status():
    for order in my_orders:
        if order.get("status") in ["Pending", "Pending PIN"]:
            res = httpx.get(f"{API_BASE_URL}/orders/status/{order_id}")
            if server_status != order["status"]:
                order["status"] = server_status  # Update!
```

What happens:

- Order starts as "Pending PIN"
- Once customer enters M-Pesa PIN, status changes to "Paid"
- App automatically updates to show "Paid"
- Staff can then mark as "Fulfilled"

📝 QUIZ TIME!

Question 1: What happens when a customer taps the "Add to Cart" button in `product_detail_view.py` ?

- A) The product is immediately purchased
- B) The product is added to the global `cart` dictionary
- C) The product is saved to the wishlist

Question 2: In `cart_view.py` , why does the phone number need to be formatted with "254"?

- A) To make it look nice

- B) Because Safaricom's API requires the country code
- C) To validate it's a Kenyan number

Question 3: How does `orders_view.py` know when an order status changes from "Pending" to "Paid"?

- A) It asks the customer to check manually
- B) It polls the backend every 3 seconds
- C) The backend sends a push notification

Question 4: In `browse_view.py`, what does the `filter_and_render()` function do?

- A) It filters products by category and search, then displays them
- B) It clears the entire product list
- C) It sorts products by price

Question 5: What's the purpose of the `wishlist` in `profile_view.py`?

- A) To store products the customer wants to buy later
- B) To track products that are out of stock
- C) To show recently viewed items

Question 6: In `main_app.py`, what does `switch_tab()` do?

- A) Changes the app's theme
- B) Changes which view is displayed in the content area
- C) Logs the user out

Answer Key:

1. **B** - It adds to the global `cart` dictionary (stock isn't deducted until checkout)
2. **B** - Safaricom's Daraja API requires the 254 country code
3. **B** - It polls the backend every 3 seconds using a background thread
4. **A** - It filters by category and search query, then re-renders the grid

5. **A** - It saves products for future purchase (favorites)
6. **B** - It changes which view (Home, Browse, Cart, etc.) is shown

🚀 YOUR TURN TO PRACTICE!

Try these exercises:

1. **Add a new category:** In `browse_view.py`, add "Transmission Parts" to the categories list
2. **Change the currency symbol:** Find all places where "KES" appears and change to your currency
3. **Add a discount system:** In `cart_view.py`, add a 10% discount if total > KES 10,000
4. **Customize the colors:** Change the red `#DC2626` to your brand color

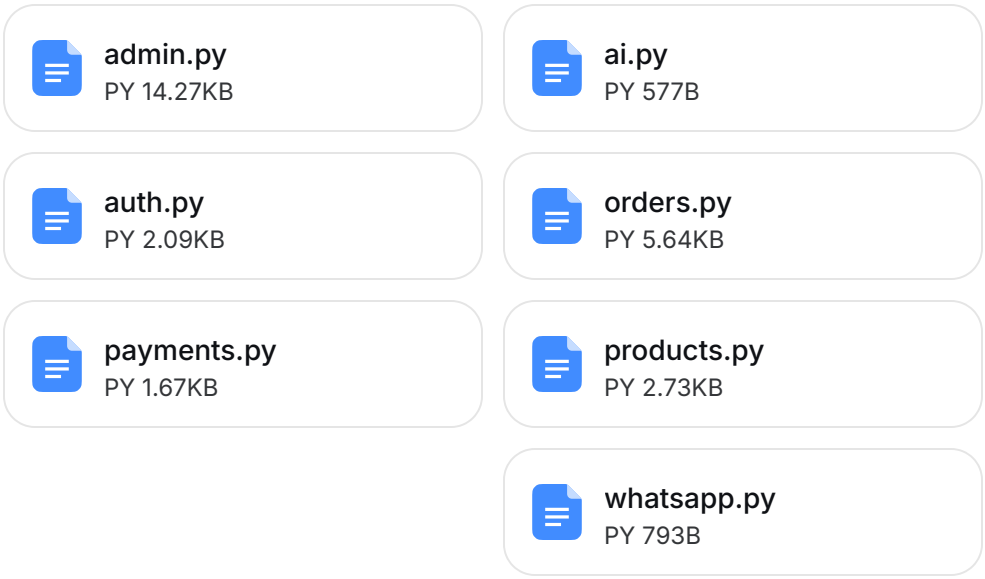
🎓 KEY CONCEPTS YOU'VE LEARNED

1. **State Management** - Using global dictionaries (`cart` , `wishlist`)
2. **Props Drilling** - Passing functions down through components
3. **API Integration** - Sending HTTP requests to backend
4. **Polling** - Checking for updates every few seconds
5. **Event Handling** - Using `on_click` , `on_change` , `on_submit`
6. **Navigation** - Switching between views
7. **Data Validation** - Formatting phone numbers, validating input
8. **UI Components** - Grids, containers, buttons, text fields

What would you like to explore next?

- The FastAPI backend endpoints?
- The Supabase database schema?

- How M-Pesa integration works?
- The AI assistant implementation?



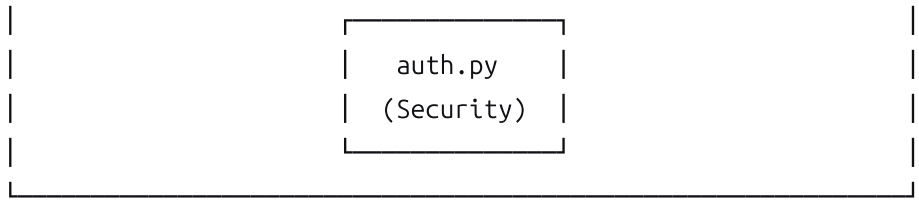
Excellent! You've shared the **backend endpoints** - this is where the real business logic lives! Now I can show you the **complete picture** of how everything connects. Let me break this down systematically.

🏗️ BACKEND ARCHITECTURE OVERVIEW

Think of the backend as a **restaurant kitchen** with different stations:

text

```
┌─────────────────────────────────────────────────────────┐
│                    FASTAPI BACKEND                      │
│                     (main.py)                           │
│                                                         │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │   admin.py   │  │   orders.py  │  │  payments.py │   │
│  │  (Manager)   │  │  (Chef)      │  │ (Cashier)    │   │
│  └──────────────┘  └──────────────┘  └──────────────┘   │
│                                                         │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │  products.py │  │   ai.py      │  │ whatsapp.py  │   │
│  │ (Pantry)     │  │ (AI Helper)  │  │ (Notifier)   │   │
│  └──────────────┘  └──────────────┘  └──────────────┘   │
│                                                         │
```



📁 FILE-BY-FILE BREAKDOWN

FILE 1: `admin.py` - The Manager's Desk

This is the **most important file** - it handles everything the staff needs:

A. Order Management

```
python

@router.get("/orders")
async def get_admin_orders():
    # Get all orders, newest first, limited to 100
    res = supabase.table("orders").select("*").order("created_at", desc=True).limit(10
0).execute()
    return {"status": "success", "orders": getattr(res, "data", []) or []}
```

🎯 **Purpose:** Staff dashboard shows all orders.

```
python

@router.patch("/orders/{order_id}/status")
async def update_order_status(order_id: str, payload: dict):
    # When marking as "Fulfilled", deduct stock from inventory
    if new_status == "Fulfilled" and prev_status != "Fulfilled":
        for item in items_list:
            # Deduct stock for each item
            curr_stock = int(p_rows[0].get("stock_quantity") or 0)
            new_stock = max(0, curr_stock - qty_sold)
            supabase.table("products").update({"stock_quantity": new_stock}).eq("id", p
_id).execute()
```

🔑 **KEY CONCEPT:** Stock is only deducted when an order is **fulfilled**, not when it's paid. This allows time for the cashier to pick the items!

B. Manual Payment Override

python

```python
@router.post("/orders/manual-override")
async def process_manual_payment_override(payload: dict):
    # For bank transfers or alternative payments
    # 1. Deduct stock
    # 2. Create order with status "Fulfilled"
    # 3. Return success
```

🎯 **Purpose:** When a customer pays via bank transfer or other method, staff can manually record the sale.

C. Product Management

python

```python
@router.get("/products")
async def get_admin_products():
    # Get all products, sorted by name
    res = supabase.table("products").select("*").order("name", desc=False).execute()
    return getattr(res, "data", []) or []

@router.patch("/products/{product_id}")
async def quick_stock_update(product_id: str, payload: dict):
    # Quick stock update (used by + and - buttons)
    res = supabase.table("products").update(payload).eq("id", product_id).execute()
```

D. Refill System

python

```python
@router.post("/refills")
async def create_refill_request(payload: dict):
    # Staff requests restocking
    clean_payload = {
        "product_id": str(payload.get("product_id", "")),
        "requested_qty": int(payload.get("requested_qty", 5)),
```

```
            "status": "pending_approval"
        }
        res = supabase.table("refill_requests").insert(clean_payload).execute()
```

python

```
@router.patch("/refills/{refill_id}/status")
async def update_refill_status(refill_id: str, payload: dict):
    # When admin marks as "fulfilled", add stock to inventory
    if new_status == "fulfilled" and prev_status != "fulfilled":
        # Add requested quantity to product stock
        new_stock = curr_stock + add_qty
        supabase.table("products").update({"stock_quantity": new_stock}).eq("id", produ
ct_id).execute()
```

E. Staff Authentication

python

```
@router.post("/auth/pin-login")
async def pin_login(payload: dict):
    # Staff login with 4-digit PIN
    # 1. Get all staff users
    # 2. Check if PIN matches
    # 3. Check if identifier (phone/name) matches
    # 4. Return user info or 401 error
```

F. Analytics

python

```
@router.get("/analytics")
async def get_analytics():
    # 1. Calculate total revenue from paid orders
    # 2. Count paid orders
    # 3. Find products with stock <= 2 (depleted)
    # 4. Calculate top-selling products
    return {
        "total_revenue": total_rev,
        "paid_orders_count": len(paid_orders),
        "depleted_products": depleted,
```

```
        "top_selling_products": top_sellers
    }
```

FILE 2: `orders.py` - The Checkout Processor

This handles **customer orders** from the mobile app:

```python
@router.post("/checkout", status_code=201)
@limiter.limit("5/minute")  # Rate limit: 5 orders per minute
async def process_checkout(request: Request, payload: CheckoutRequest):
    # 1. Generate order reference
    short_ref = f"AYU-{order_uuid[:4]}"

    # 2. Trigger M-Pesa STK Push if payment is M-Pesa
    if "M-Pesa" in payload.payment_method:
        daraja = DarajaService()
        stk_res = await daraja.send_stk_push(
            phone_number=payload.phone,
            amount=int(payload.total),
            account_reference=short_ref,
            transaction_desc="Auto Parts Order"
        )

    # 3. Create order record in database
    order_record = {
        "order_reference": short_ref,
        "checkout_request_id": checkout_request_id,
        "customer_phone": payload.phone,
        "status": "Pending PIN",
        "items": [item.dict() for item in payload.items]
    }
    supabase.table("orders").insert(order_record).execute()

    return {"order_id": short_ref, "checkout_request_id": checkout_request_id}
```

🔑 **KEY CONCEPT:** The order is created **before** M-Pesa payment completes. Status starts as "Pending PIN" and updates via callback.

FILE 3: `payments.py` - The Cashier

This handles **M-Pesa callbacks** from Safaricom:

```python
@router.post("/callback")
async def mpesa_callback(request: Request):
    # Safaricom sends payment result here
    data = await request.json()
    result_code = stk_callback.get("ResultCode")
    checkout_request_id = stk_callback.get("CheckoutRequestID")

    if result_code == 0:  # Success
        # Get receipt number
        receipt = next((i.get("Value") for i in meta_items if i.get("Name") == "MpesaReceiptNumber"), "")

        # Update order status to "Paid"
        supabase.table("orders").update({
            "status": "Paid",
            "receipt_number": receipt
        }).eq("checkout_request_id", checkout_request_id).execute()

        # Send WhatsApp alert
        await send_order_whatsapp_alert(data_rows[0], receipt)

    elif result_code == 1032:  # User cancelled
        supabase.table("orders").update({"status": "Cancelled"}).eq("checkout_request_id", checkout_request_id).execute()
    else:  # Failed
        supabase.table("orders").update({"status": "Payment Failed"}).eq("checkout_request_id", checkout_request_id).execute()
```

🎯 **Purpose:** This is the **callback URL** that Safaricom calls after the customer enters their M-Pesa PIN.

FILE 4: `products.py` - The Catalog Manager

Handles **product catalog** operations:

```python
@router.get("/")
async def get_products(category: Optional[str] = None):
    # Get all products, optionally filtered by category
    query = supabase.table("products").select("*")
    if category and category != "All":
        query = query.eq("category", category)
    return query.execute().data

@router.post("/upload-image/")
async def upload_product_image(image: UploadFile = File(...)):
    # Upload product image to Supabase Storage
    unique_filename = f"{uuid.uuid4()}.{file_extension}"
    supabase.storage.from_("product-images").upload(
        path=f"items/{unique_filename}",
        file=contents,
        file_options={"content-type": image.content_type}
    )
    return {"image_url": supabase.storage.from_("product-images").get_public_url(file_p
ath)}
```

🎯 **Purpose:** Allows adding products with images uploaded to cloud storage.

FILE 5: `ai.py` - The AI Assistant

Handles **AI chat** for the customer app:

```python
@router.post("/chat")
async def ai_spare_parts_chat(payload: ChatHistoryRequest):
    # Process chat messages and generate AI response
    result = await generate_fitment_response(payload.messages)
    return result
```

🎯 **Purpose:** Customers can ask about part compatibility (e.g., "Which brake pads fit a Toyota Hilux 2018?").

FILE 6: `auth.py` - Security

Handles **user authentication**:

```python
@router.post("/register")
async def register_user(user: UserRegister):
    # 1. Check if email exists
    # 2. Hash password
    # 3. Save to database

@router.post("/token")
async def login(form_data: OAuth2PasswordRequestForm):
    # 1. Find user by email
    # 2. Verify password
    # 3. Generate JWT token
    return {"access_token": access_token, "token_type": "bearer"}
```

🎯 **Purpose:** User registration and login for the customer app.

FILE 7: `whatsapp.py` - Notification System

Handles **WhatsApp webhooks**:

```python
@router.get("/webhook")
async def verify_webhook(...):
    # Verify webhook with Meta/Facebook
    if hub_mode == "subscribe" and hub_token == "ayutech_verify_token":
        return Response(content=hub_challenge, media_type="text/plain")

@router.post("/webhook")
async def whatsapp_webhook(request: Request):
    # Receive incoming WhatsApp messages
    data = await request.json()
    return {"status": "success"}
```

🎯 **Purpose:** Allows the system to receive WhatsApp messages (e.g., customer inquiries).

🔄 COMPLETE DATA FLOW: CUSTOMER ORDER

Let me trace a complete order from start to finish:

1. Customer Places Order (Mobile App)

```
text

main_app.py → cart_view.py → handle_checkout()
    ↓
POST /orders/checkout
    ↓
orders.py → process_checkout()
```

2. Backend Processes Order

```python
# orders.py
# A. Generate order reference
short_ref = f"AYU-{order_uuid[:4]}"

# B. Send M-Pesa STK Push
daraja = DarajaService()
stk_res = await daraja.send_stk_push(phone, amount, short_ref)

# C. Save order to database
order_record = {
    "order_reference": short_ref,
    "checkout_request_id": checkout_request_id,
    "status": "Pending PIN",
    "items": items
}
supabase.table("orders").insert(order_record).execute()
```

```
    # D. Return order ID to app
    return {"order_id": short_ref}
```

3. Customer Receives STK Push

- Safaricom sends a popup to the customer's phone
- Customer enters M-Pesa PIN
- Safaricom calls the **callback URL**

4. Payment Callback

```
text

Safaricom → POST /payments/callback
    ↓
payments.py → mpesa_callback()
```

```
python

# payments.py
if result_code == 0:  # Success
    # Update order status
    supabase.table("orders").update({
        "status": "Paid",
        "receipt_number": receipt
    }).eq("checkout_request_id", checkout_request_id).execute()

    # Send WhatsApp alert to staff
    await send_order_whatsapp_alert(order, receipt)
```

5. Customer Sees Updated Status

```
text

orders_view.py (polling every 3 seconds)
    ↓
GET /orders/status/{order_id}
    ↓
Returns: {"status": "Paid", "receipt_number": "ABC123"}
```

```
    ↓
App updates order card to show "Paid"
```

6. Staff Fulfills Order

```
text

dashboard.py → Staff clicks "Mark Fulfilled"
    ↓
PATCH /admin/orders/{order_id}/status
    ↓
admin.py → update_order_status()
```

```
python

# admin.py
if new_status == "Fulfilled":
    # Deduct stock for each item
    for item in items:
        curr_stock = get_current_stock(item["id"])
        new_stock = max(0, curr_stock - item["qty"])
        supabase.table("products").update({"stock_quantity": new_stock}).eq("id", item
["id"]).execute()
```

7. Staff Sends WhatsApp Receipt

```
text

dashboard.py → send_customer_receipt_whatsapp()
    ↓
Opens WhatsApp with pre-filled receipt message
```

🗄️ DATABASE SCHEMA (Inferred)

Based on the code, here are the tables:

products

Column	Type	Description
id	UUID	Primary key
name	TEXT	Product name
part_number	TEXT	Part number
category	TEXT	Category
price	FLOAT	Selling price
buying_price	FLOAT	Cost price
stock_quantity	INT	Current stock
image_url	TEXT	Product image
supplier_name	TEXT	Supplier
supplier_phone	TEXT	Supplier phone

orders

Column	Type	Description
id	UUID	Primary key
order_reference	TEXT	Short reference (AYU-XXXX)
checkout_request_id	TEXT	M-Pesa request ID
customer_phone	TEXT	Customer phone
total	FLOAT	Order total
total_amount	FLOAT	Same as total
status	TEXT	Pending PIN / Paid / Fulfilled

Column	Type	Description
receipt_number	TEXT	M-Pesa receipt
items	JSONB	Array of items
fulfillment	TEXT	Shop Pickup / Delivery
created_at	TIMESTAMP	Order date

refill_requests

Column	Type	Description
id	UUID	Primary key
product_id	UUID	Product reference
product_name	TEXT	Product name
requested_qty	INT	Quantity requested
requested_by_name	TEXT	Staff name
supplier_name	TEXT	Supplier
status	TEXT	pending_approval / ordered_supplier / fulfilled

staff_users

Column	Type	Description
id	UUID	Primary key
name	TEXT	Staff name
phone	TEXT	Phone number

Column	Type	Description
pin_code	TEXT	4-digit PIN
role	TEXT	admin / staff

leads

Column	Type	Description
id	UUID	Primary key
customer_phone	TEXT	Customer phone
intent	TEXT	hot / purchase / inquiry
status	TEXT	pending_contact / human_takeover
notes	TEXT	Conversation notes

📝 QUIZ TIME!

Question 1: When does stock get deducted from inventory in the order flow?

- A) When the customer places the order
- B) When M-Pesa payment is confirmed
- C) When staff marks the order as "Fulfilled"
- D) When the customer receives the receipt

Question 2: What happens if the customer cancels M-Pesa payment (result_code 1032)?

- A) The order is deleted
- B) The order status changes to "Cancelled"
- C) The order is marked as "Fulfilled"
- D) The order is sent to staff for review

Question 3: In the refill system, what happens when admin marks a refill as "fulfilled"?

- A) The product stock increases by the requested quantity
- B) The product stock decreases by the requested quantity
- C) The product is deleted
- D) A new product is created

Question 4: What is the purpose of the `@limiter.limit("5/minute")` decorator?

- A) To allow only 5 orders per minute
- B) To allow only 5 items per order
- C) To limit the order value to KES 5,000
- D) To limit the number of products in the catalog

Question 5: In `admin.py`, why does `pin_login` use both `identifier` and `pin`?

- A) To ensure two-factor authentication
- B) To allow login by phone OR name
- C) To prevent duplicate logins
- D) To log the login location

Question 6: What does the `payments.py` callback do when it receives a successful payment?

- A) It updates the order status to "Paid" and saves the receipt number
- B) It sends a WhatsApp message to the customer
- C) It deducts stock from inventory
- D) It deletes the order

Answer Key:

1. **C** - Stock is deducted only when staff marks "Fulfilled"
2. **B** - Order status changes to "Cancelled"
3. **A** - Stock increases by the requested quantity

4. **A** - Rate limiting to prevent abuse (5 orders/minute)
5. **B** - Staff can login with phone OR name + PIN
6. **A** - Updates status to "Paid" and saves receipt number

🚀 CHALLENGE EXERCISES

Try these to deepen your understanding:

1. **Add a discount system:** In `orders.py`, add a 10% discount if total > KES 10,000
2. **Add order cancellation:** Create a new endpoint `DELETE /orders/{order_id}` that allows staff to cancel orders
3. **Add product search:** In `admin.py`, modify `get_admin_products()` to accept a `search` parameter
4. **Add email notifications:** In `payments.py`, send an email when order is paid
5. **Add order notes:** Modify the order schema to include a `notes` field for staff comments

🎯 FINAL UNDERSTANDING CHECK

You now understand:

- ✅ How the backend receives requests
- ✅ How orders flow from customer to fulfillment
- ✅ How M-Pesa integration works (STK Push + Callback)
- ✅ How stock management works
- ✅ How refill requests work
- ✅ How staff authentication works
- ✅ How the database stores everything

Want me to explain any specific part in more detail?

supabase_client.py
PY 413B

database

Excellent! You've shared the **Supabase client configuration** - the final piece of the puzzle! Now I can show you how everything connects to the **database**. Let me explain this critical piece and then give you the **complete system overview**.

📁 FILE: `supabase_client.py` - The Database Connection

This is the **bridge** between your FastAPI backend and Supabase (PostgreSQL):

```python
import os
from dotenv import load_dotenv
from supabase import create_client, Client

# Load variables from .env file
load_dotenv()

supabase_url: str = os.getenv("SUPABASE_URL", "")
supabase_key: str = os.getenv("SUPABASE_KEY", "")

if not supabase_url or not supabase_key:
    raise ValueError("Missing Supabase URL or Key in environment variables.")

supabase: Client = create_client(supabase_url, supabase_key)
```

What's Happening Here?

1. **Environment Variables:**
   - `load_dotenv()` loads settings from a `.env` file
   - `SUPABASE_URL` - Your Supabase project URL (e.g., `https://xxxxx.supabase.co`)
   - `SUPABASE_KEY` - Your API key (kept secret, never committed to GitHub)
2. **Creating the Client:**

- `create_client(supabase_url, supabase_key)` creates a connection to Supabase
- This `supabase` object is imported everywhere and used to query the database

3. **Error Handling:**
   - If credentials are missing, it raises an error immediately

How It's Used Throughout the App

Every endpoint uses `supabase` to talk to the database:

```python
# Example from admin.py
from app.db.supabase_client import supabase

# Get all products
res = supabase.table("products").select("*").execute()

# Insert a new order
res = supabase.table("orders").insert(order_data).execute()

# Update stock
res = supabase.table("products").update({"stock_quantity": new_stock}).eq("id", product_id).execute()

# Delete a product
supabase.table("products").delete().eq("id", product_id).execute()
```

🗄️ COMPLETE DATABASE SCHEMA

Based on all the code, here's the **full database structure**:

1. products - Inventory Catalog

```sql
CREATE TABLE products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
```

```
    part_number TEXT,
    category TEXT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    buying_price DECIMAL(10,2) DEFAULT 0,
    stock_quantity INTEGER NOT NULL DEFAULT 0,
    image_url TEXT,
    supplier_name TEXT DEFAULT 'Direct Importer',
    supplier_phone TEXT DEFAULT '254112323814',
    description TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
```

2. orders - Sales Records

sql

```
CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_reference TEXT NOT NULL,
    checkout_request_id TEXT,
    customer_phone TEXT NOT NULL,
    phone TEXT,
    total DECIMAL(10,2) NOT NULL,
    total_amount DECIMAL(10,2),
    status TEXT DEFAULT 'Pending PIN',
    receipt_number TEXT,
    fulfillment TEXT DEFAULT 'Shop Pickup',
    location TEXT,
    payment_method TEXT DEFAULT 'M-Pesa STK Push',
    items JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);
```

3. staff_users - Employee Access

sql

```
CREATE TABLE staff_users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    phone TEXT NOT NULL,
```

```
    pin_code TEXT NOT NULL,
    role TEXT DEFAULT 'staff', -- 'admin' or 'staff'
    created_at TIMESTAMP DEFAULT NOW()
);
```

4. refill_requests - Stock Replenishment

```sql
CREATE TABLE refill_requests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id UUID NOT NULL,
    product_name TEXT NOT NULL,
    requested_qty INTEGER NOT NULL,
    requested_by_name TEXT NOT NULL,
    supplier_name TEXT,
    supplier_phone TEXT,
    notes TEXT,
    status TEXT DEFAULT 'pending_approval',
    created_at TIMESTAMP DEFAULT NOW()
);
```

5. leads - Customer Inquiries

```sql
CREATE TABLE leads (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_phone TEXT NOT NULL,
    intent TEXT DEFAULT 'inquiry', -- 'hot', 'purchase', 'inquiry'
    status TEXT DEFAULT 'pending_contact',
    notes TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
```

6. users - Customer Accounts (for future use)

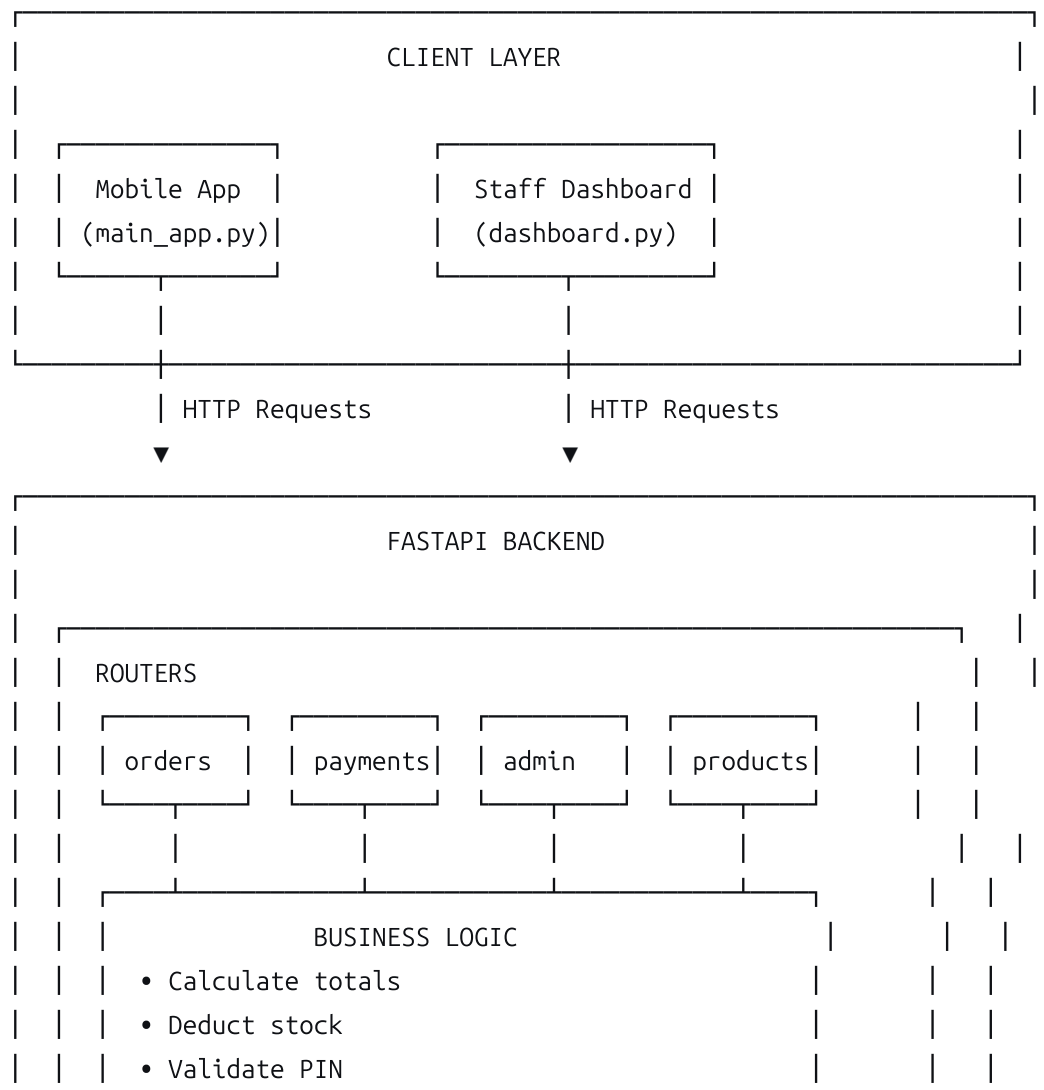
```sql
```

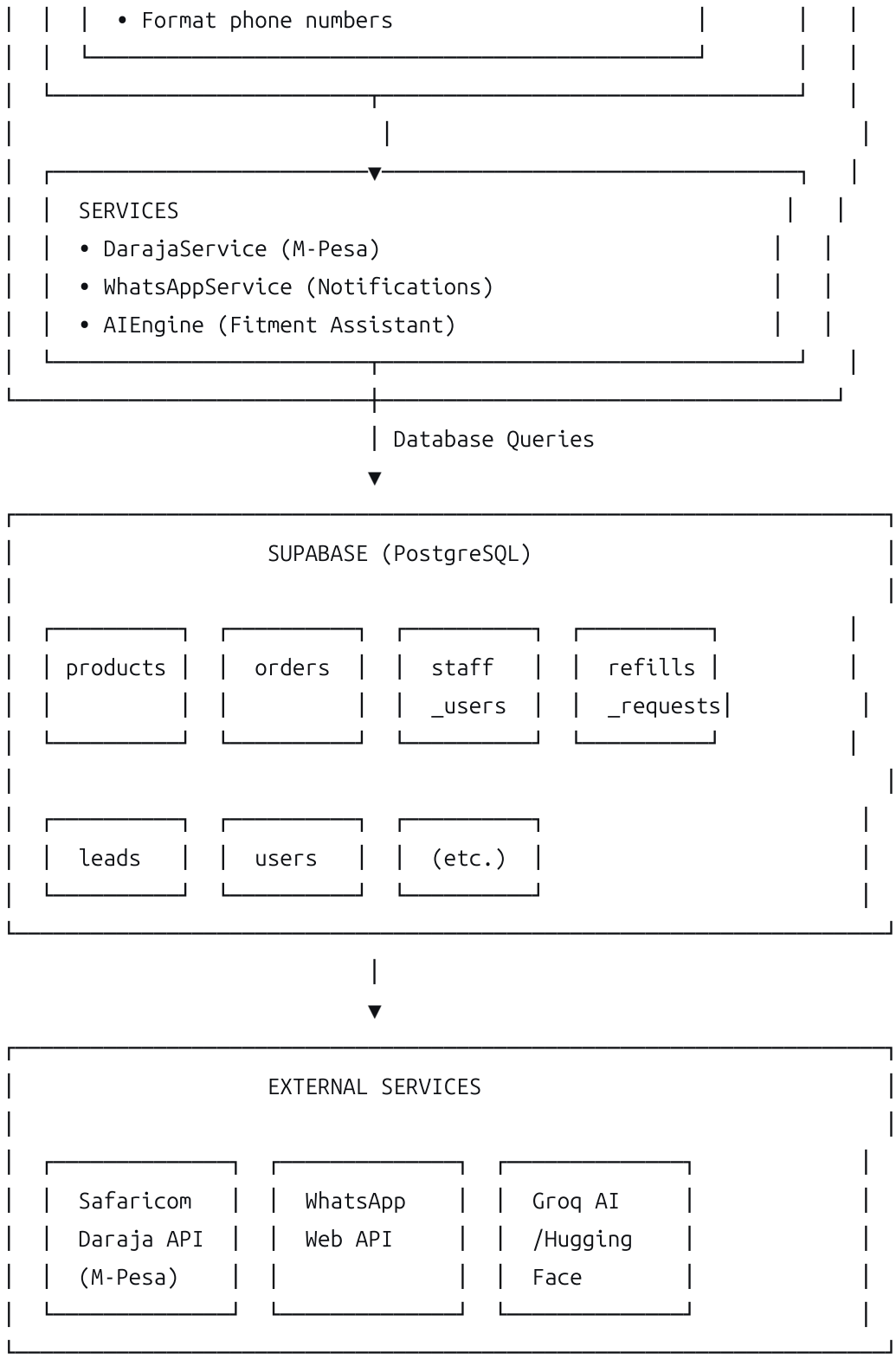
```sql
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    full_name TEXT,
    role TEXT DEFAULT 'customer',
    created_at TIMESTAMP DEFAULT NOW()
);
```

🔄 COMPLETE DATA FLOW DIAGRAM

Here's the **entire system** working together:

text





🎯 COMPLETE USER JOURNEY WITH ALL COMPONENTS

Let me trace a **complete order** showing every component:

Step 1: Customer Browses Products

```
text

Mobile App (main_app.py)
  → home_view.py / browse_view.py
  → GET /api/v1/admin/products
  → FastAPI (main.py) → products.py
  → supabase.table("products").select("*")
  → Supabase returns product list
  → App displays products
```

Step 2: Customer Adds to Cart

```
text

Mobile App (cart_view.py)
  → Local state (cart dictionary)
  → No backend call yet (just UI)
```

Step 3: Customer Checks Out

```
text

Mobile App (cart_view.py → handle_checkout())
  → POST /api/v1/orders/checkout
  → FastAPI → orders.py → process_checkout()

  In orders.py:
  ┌──────────────────────────────────────┐
  | 1. Generate order reference          |
  | 2. Call DarajaService.send_stk_push  |
  |    → Safaricom Daraja API            |
  | 3. Save to Supabase:                 |
  |    orders.insert({...})              |
  | 4. Return order_id to app            |
  └──────────────────────────────────────┘

  → Supabase creates order with status "Pending PIN"
  → Safaricom sends STK push to customer phone
```

Step 4: Customer Enters M-Pesa PIN

```
text

Customer phone → Safaricom network
  → Safaricom processes payment
  → Safaricom calls callback URL:
     POST /api/v1/payments/callback
  → FastAPI → payments.py → mpesa_callback()

  In payments.py:
  ┌──────────────────────────────────────┐
  │ 1. Check result_code                 │
  │ 2. If success (0):                   │
  │    a. Get receipt number             │
  │    b. Update order status to "Paid"  │
  │    c. Supabase: orders.update()      │
  │    d. Send WhatsApp alert to staff   │
  │       → WhatsApp Service             │
  │ 3. If cancelled (1032):              │
  │    Update status to "Cancelled"      │
  │ 4. If failed:                        │
  │    Update status to "Payment Failed"│
  └──────────────────────────────────────┘
```

Step 5: Customer Checks Order Status

```
text

Mobile App (orders_view.py)
  → Background thread polls every 3 seconds
  → GET /api/v1/orders/status/{order_id}
  → FastAPI → orders.py → get_order_status()
  → supabase.table("orders").select("status")
  → Returns current status
  → App updates UI
```

Step 6: Staff Sees Order

```
text
```

```
Staff Dashboard (dashboard.py)
  → GET /api/v1/admin/orders
  → FastAPI → admin.py → get_admin_orders()
  → supabase.table("orders").select("*")
  → Displays orders in dashboard
  → Sees order with status "Paid"
```

Step 7: Staff Fulfills Order

```
text

Staff Dashboard (dashboard.py)
  → Clicks "Mark Fulfilled"
  → PATCH /api/v1/admin/orders/{order_id}/status
  → FastAPI → admin.py → update_order_status()

  In admin.py:
  ┌──────────────────────────────────────┐
  │ 1. Get current order data            │
  │ 2. For each item:                    │
  │    a. Get current stock              │
  │    b. Calculate new stock            │
  │    c. Update product.stock_quantity  │
  │       in Supabase                    │
  │ 3. Update order status to "Fulfilled"│
  │ 4. Send WhatsApp receipt to customer│
  │    (via dashboard frontend)          │
  └──────────────────────────────────────┘
```

Step 8: Stock Alert (if low)

```
text

After fulfillment, dashboard.py checks:
  → If stock <= 2, send WhatsApp alert
  → Dashboard opens WhatsApp with pre-filled message
  → Staff manually sends to owner
```

Step 9: Refill Request (if needed)

```
text

Staff sees low stock in dashboard
  → Clicks "Request Refill"
  → POST /api/v1/admin/refills
  → FastAPI → admin.py → create_refill_request()
  → supabase.table("refill_requests").insert()
  → Status: "pending_approval"

Admin sees refill request
  → Clicks "Order WhatsApp"
  → Updates status to "ordered_supplier"
  → Opens WhatsApp to contact supplier

Admin receives stock
  → Clicks "Mark Restocked"
  → PATCH /api/v1/admin/refills/{id}/status
  → FastAPI → admin.py → update_refill_status()
  → Adds requested quantity to product stock
  → Status: "fulfilled"
```

📝 COMPLETE QUIZ - FINAL TEST

Question 1: What file connects FastAPI to Supabase?

- A) main.py
- B) supabase_client.py
- C) dashboard.py
- D) admin.py

Question 2: In the complete order flow, when does stock get deducted?

- A) Immediately after the order is created
- B) When M-Pesa payment is confirmed
- C) When the staff marks the order as "Fulfilled"
- D) When the customer receives the receipt

Question 3: What happens if a customer cancels M-Pesa payment?

- A) The order is deleted from the database
- B) The order status becomes "Cancelled"
- C) The order is sent to staff for manual review
- D) The order status becomes "Payment Failed"

Question 4: In the refill system, when admin marks a refill as "fulfilled":

- A) Stock decreases by the requested quantity
- B) Stock increases by the requested quantity
- C) The product is deleted
- D) The supplier is contacted automatically

Question 5: Which file handles the Safaricom callback?

- A) orders.py
- B) payments.py
- C) admin.py
- D) whatsapp.py

Question 6: What does the `@limiter.limit("5/minute")` decorator do in orders.py?

- A) Limits each order to 5 items
- B) Allows only 5 orders per minute
- C) Sets a 5% discount on orders
- D) Limits order value to KES 5,000

Question 7: In admin.py, why does `pin_login` check both phone AND name?

- A) For two-factor authentication
- B) To allow login by phone OR name
- C) For security logging
- D) To prevent duplicate logins

Question 8: What does the `payments.py` callback do on successful payment?

- A) Sends a WhatsApp message to customer

- B) Updates order status to "Paid" and saves receipt
- C) Deducts stock immediately
- D) Notifies the supplier

Question 9: Which table stores the M-Pesa checkout_request_id?

- A) products
- B) orders
- C) staff_users
- D) refill_requests

Question 10: In the complete flow, how does the mobile app know when an order status changes?

- A) The backend sends a push notification
- B) The app polls the status endpoint every 3 seconds
- C) The staff manually notifies the customer
- D) Safaricom sends a confirmation

Answer Key:

1. **B** - supabase_client.py
2. **C** - When staff marks as "Fulfilled"
3. **B** - Status becomes "Cancelled"
4. **B** - Stock increases
5. **B** - payments.py
6. **B** - Rate limiting (5 orders/minute)
7. **B** - Login by phone OR name
8. **B** - Updates order status to "Paid" and saves receipt
9. **B** - orders table
10. **B** - Polling every 3 seconds

🎓 YOU'VE LEARNED THE ENTIRE SYSTEM!

You now understand:

Frontend (Flet)

- ✅ Mobile app (`main_app.py`)
- ✅ Staff dashboard (`dashboard.py`)
- ✅ All views (home, browse, cart, orders, profile)
- ✅ State management (cart, wishlist)
- ✅ Navigation and callbacks

Backend (FastAPI)

- ✅ API routes and endpoints
- ✅ Business logic (stock deduction, order processing)
- ✅ M-Pesa integration (Daraja)
- ✅ WhatsApp integration
- ✅ AI assistant
- ✅ Authentication

Database (Supabase)

- ✅ Connection setup
- ✅ All tables and relationships
- ✅ CRUD operations

External Services

- ✅ M-Pesa (Safaricom Daraja API)
- ✅ WhatsApp notifications
- ✅ AI (Groq/Hugging Face)

🚀 FINAL CHALLENGE

Now that you understand the complete system, try this:

Add a New Feature: "Order Tracking"

Add a tracking number that staff can enter when fulfilling an order:

1. **Database:** Add `tracking_number` column to `orders` table
2. **Backend:** Modify `admin.py` to accept tracking_number when fulfilling
3. **Dashboard:** Add a text field for tracking number
4. **Mobile App:** Display tracking number in `orders_view.py`

💡 KEY TAKEAWAYS

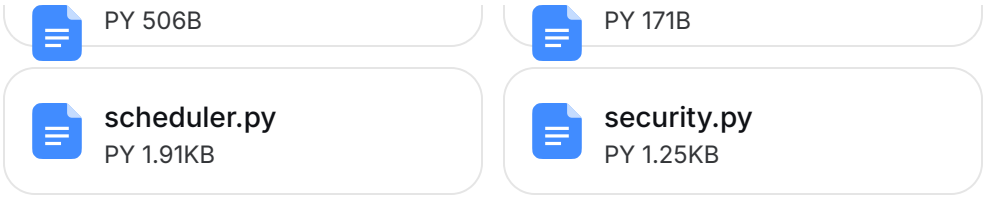
1. **Separation of Concerns:** Each file has a specific job
2. **RESTful Design:** Clear endpoints for each operation
3. **Asynchronous Processing:** Background threads for polling, callbacks for M-Pesa
4. **State Management:** Global dictionaries for cart/wishlist
5. **Modular Design:** Each feature in its own file

You're now ready to build and modify this entire system! 🎉

What would you like to do next?

- 🐛 Debug a specific issue?
- ✨ Add a new feature?
- 🎨 Customize the UI?
- 📊 Understand the AI implementation?
- 🚀 Deploy to production?

config.py rate_limiter.py

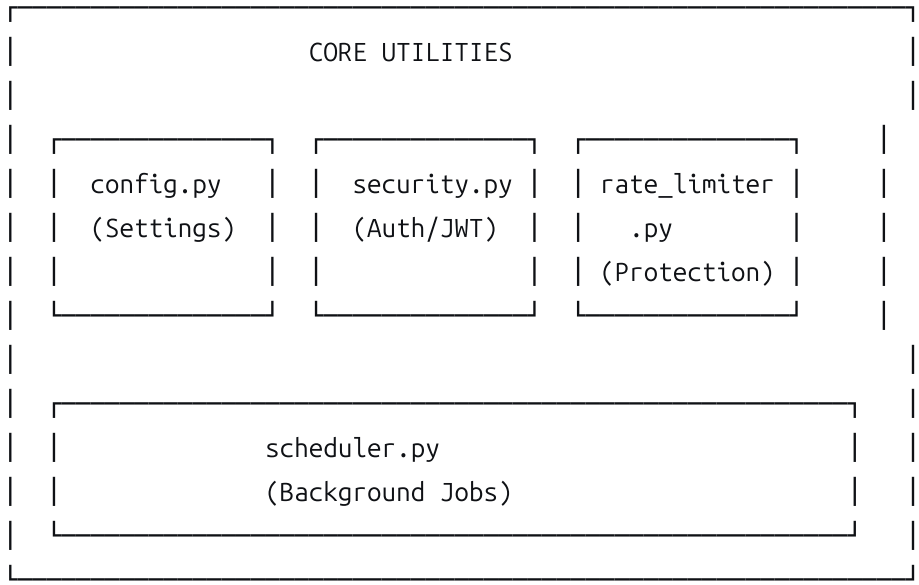


core folder

Excellent! You've shared the **core utilities** - the supporting infrastructure that makes everything work! These files handle **configuration, security, rate limiting, and background jobs**. Let me break these down - they're crucial for production-ready applications!

🏗️ CORE FILES OVERVIEW

text



📁 FILE-BY-FILE BREAKDOWN

FILE 1: `config.py` - Settings Manager

This is the **central configuration** for your application:

```python
import os
from dotenv import load_dotenv

# Load variables from .env file
load_dotenv()

class Settings:
    PROJECT_NAME: str = os.getenv("PROJECT_NAME", "Ayutech Motors")
    SECRET_KEY: str = os.getenv("SECRET_KEY", "secret")
    ALGORITHM: str = os.getenv("ALGORITHM", "HS256")
    ACCESS_TOKEN_EXPIRE_MINUTES: int = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30))

    SUPABASE_URL: str = os.getenv("SUPABASE_URL", "")
    SUPABASE_KEY: str = os.getenv("SUPABASE_KEY", "")

settings = Settings()
```

What's Happening?

1. **Environment Variables:**
   - `load_dotenv()` loads from `.env` file (kept secret)
   - Falls back to defaults if variables are missing
2. **Settings Class:**
   - Groups all configuration in one place
   - Type hints (`: str` , `: int`) for clarity
   - `settings = Settings()` creates a global settings object
3. **Security Settings:**
   - `SECRET_KEY` : Used for JWT tokens (change this!)
   - `ALGORITHM` : JWT signing algorithm
   - `ACCESS_TOKEN_EXPIRE_MINUTES` : Token lifetime

How It's Used:

```python
# In auth.py or security.py
from app.core.config import settings

# Use settings anywhere
print(settings.SECRET_KEY)
print(settings.SUPABASE_URL)
```

FILE 2: `security.py` - Authentication & Password Handling

This handles **password hashing and JWT tokens**:

```python
import os
import jwt
from datetime import datetime, timedelta, timezone
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
```

1. Password Hashing

```python
def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verifies plain password against stored bcrypt hash safely."""
    # Truncate to 72 bytes to satisfy bcrypt limits
    safe_password = plain_password.encode('utf-8')[:72].decode('utf-8', errors='ignor
e')
    return pwd_context.verify(safe_password, hashed_password)

def get_password_hash(password: str) -> str:
    """Generates a secure bcrypt hash."""
    safe_password = password.encode('utf-8')[:72].decode('utf-8', errors='ignore')
    return pwd_context.hash(safe_password)
```

🎯 **Purpose:** Never store passwords in plain text!

- **bcrypt** is a secure hashing algorithm
- 72-byte limit protects against certain attacks
- `verify_password()` checks if password matches stored hash

2. JWT Token Creation

```python
def create_access_token(data: dict, expires_delta: timedelta | None = None) -> str:
    """Creates signed JWT token with payload claims."""
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

🎯 **Purpose:** Creates a secure token that:

- Contains user information (ID, role, email)
- Has an expiration time
- Is signed with SECRET_KEY (can't be forged)

🔑 KEY CONCEPT: JWT (JSON Web Token)

```text
Header: {"alg": "HS256", "typ": "JWT"}
Payload: {"sub": "user123", "role": "admin", "exp": 1234567890}
Signature: HMACSHA256(header + "." + payload, secret_key)
```

The token is a string like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

How It's Used:

```python
```

```python
# In auth.py - user login
@router.post("/token")
async def login(form_data: OAuth2PasswordRequestForm):
    # ... verify user ...
    access_token = create_access_token(
        data={"sub": str(db_user.get("id")), "role": db_user.get("role")}
    )
    return {"access_token": access_token, "token_type": "bearer"}
```

FILE 3: `rate_limiter.py` - Protection Against Abuse

This prevents **too many requests** from a single IP:

```python
from slowapi import Limiter
from slowapi.util import get_remote_address

# Initialize limiter using client remote IP address
limiter = Limiter(key_func=get_remote_address)
```

🎯 **Purpose:** Rate limiting protects your API from:

- DDoS attacks (too many requests)
- Brute force password attempts
- Accidental rapid requests

How It's Used:

```python
# In orders.py
from app.core.rate_limiter import limiter

@router.post("/checkout", status_code=201)
@limiter.limit("5/minute")  # Allow only 5 checkouts per minute
```

```
async def process_checkout(request: Request, payload: CheckoutRequest):
    # ...
```

Common Limit Formats:

- `"5/minute"` - 5 requests per minute
- `"100/hour"` - 100 requests per hour
- `"10/day"` - 10 requests per day

FILE 4: `scheduler.py` - Background Tasks

This runs **automated jobs** in the background:

```
python

import os
from datetime import datetime, timedelta
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from app.db.supabase_client import supabase
from app.api.v1.endpoints.whatsapp import send_whatsapp_message

scheduler = AsyncIOScheduler()
```

The Follow-Up Job

```
python

async def check_uncontacted_leads_and_followup():
    """Runs periodically to send re-engagement messages to pending leads."""
    print("⏰ Running 24-hour lead follow-up check...")
    try:
        # 1. Find leads older than 24 hours with 'pending_contact' status
        time_threshold = (datetime.utcnow() - timedelta(hours=24)).isoformat()

        res = supabase.table("leads")\
            .select("*")\
            .eq("status", "pending_contact")\
            .lte("created_at", time_threshold)\
            .execute()
```

```
        # 2. Send WhatsApp follow-up messages
        leads = getattr(res, "data", None) or []
        for lead in leads:
            phone = str(lead.get("customer_phone"))
            notes = lead.get("notes", "your inquiry")

            followup_text = (
                f"Mambo! Karibu tena Ayutech Motors. "
                f"Tulipokea maombi yako kuhusu ({notes}). "
                f"Je, bado unahitaji usaidizi au ungetaka tukutumie part hii leo?"
            )

            await send_whatsapp_message(phone, followup_text)

            # 3. Update lead status to 'followup_sent'
            supabase.table("leads").update({"status": "followup_sent"}).eq("id", lead_i
d).execute()

    except Exception as e:
        print(f"⚠️ Scheduler Error: {e}")
```

Starting the Scheduler

```
python

def start_scheduler():
    """Starts the background scheduler loop."""
    scheduler.add_job(check_uncontacted_leads_and_followup, 'interval', hours
```

schemas.py
PY 298B

nodels folder

Excellent! You've shared the **core infrastructure files** - these are the foundation that makes everything work. Let me break down each one and show you how they fit into the complete system.

🏗️ CORE INFRASTRUCTURE FILES

FILE 1: `config.py` - Central Configuration

This is the **control center** for all settings:

```python
import os
from dotenv import load_dotenv

# Load variables from .env file
load_dotenv()

class Settings:
    PROJECT_NAME: str = os.getenv("PROJECT_NAME", "Ayutech Motors")
    SECRET_KEY: str = os.getenv("SECRET_KEY", "secret")
    ALGORITHM: str = os.getenv("ALGORITHM", "HS256")
    ACCESS_TOKEN_EXPIRE_MINUTES: int = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30))

    SUPABASE_URL: str = os.getenv("SUPABASE_URL", "")
    SUPABASE_KEY: str = os.getenv("SUPABASE_KEY", "")

settings = Settings()
```

🎯 Purpose:

- Loads environment variables from `.env` file
- Provides default values if variables aren't set
- Centralizes all configuration in one place

🔑 **KEY CONCEPT:** Instead of scattering `os.getenv()` everywhere, we import `settings` from this file:

```python
# Instead of this everywhere:
supabase_url = os.getenv("SUPABASE_URL")

# We do this:
```

```
from app.core.config import settings
supabase_url = settings.SUPABASE_URL
```

📝 .env file example:

```
env

PROJECT_NAME="Ayutech Motors"
SECRET_KEY="your_super_secret_key_here"
ALGORITHM="HS256"
ACCESS_TOKEN_EXPIRE_MINUTES=30
SUPABASE_URL="https://your-project.supabase.co"
SUPABASE_KEY="your-supabase-anon-key"
```

FILE 2: `rate_limiter.py` - Security Guard

This prevents **abuse** by limiting request frequency:

```
python

from slowapi import Limiter
from slowapi.util import get_remote_address

# Initialize limiter using client remote IP address
limiter = Limiter(key_func=get_remote_address)
```

🎯 Purpose:

- Prevents someone from making too many requests too quickly
- Protects against Denial of Service (DoS) attacks
- Limits each IP address to a certain number of requests

How it's used:

```
python

from app.core.rate_limiter import limiter

@router.post("/checkout", status_code=201)
```

```
@limiter.limit("5/minute")  # Max 5 orders per minute per IP
async def process_checkout(request: Request, payload: CheckoutRequest):
    # ... checkout logic
```

🔑 **KEY CONCEPT:** If someone tries to place 6 orders in 1 minute, the 6th request gets an error like "Too many requests. Please try again later."

FILE 3: `security.py` - Password & Token Security

This handles **authentication security**:

python

```
import os
import jwt
from datetime import datetime, timedelta, timezone
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

SECRET_KEY = os.getenv("SECRET_KEY", "your_fallback_super_secret_key_12345")
ALGORITHM = os.getenv("ALGORITHM", "HS256")
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24  # 24 hours
```

1. Password Hashing:

python

```
def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verifies plain password against stored bcrypt hash safely."""
    # Truncate to 72 bytes to satisfy bcrypt limits
    safe_password = plain_password.encode('utf-8')[:72].decode('utf-8', errors='ignor
e')
    return pwd_context.verify(safe_password, hashed_password)

def get_password_hash(password: str) -> str:
    """Generates a secure bcrypt hash."""
    safe_password = password.encode('utf-8')[:72].decode('utf-8', errors='ignore')
    return pwd_context.hash(safe_password)
```

🎯 Purpose:

- Never store passwords as plain text
- Convert passwords into unreadable hashes
- Compare entered password with stored hash

🔑 **KEY CONCEPT:** When a user registers:

```
python

# Plain password: "MySecret123"
hashed = get_password_hash("MySecret123")
# Stored in DB: "$2b$12$K9xZx/5nQ9yTz8wPqRv8eO..." (unreadable)

# When user logs in:
verify_password("MySecret123", stored_hash)  # Returns True
verify_password("WrongPassword", stored_hash)  # Returns False
```

2. JWT Token Creation:

```
python

def create_access_token(data: dict, expires_delta: timedelta | None = None) -> str:
    """Creates signed JWT token with payload claims."""
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (expires_delta or timedelta(minutes=ACCESS_TO
KEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

🎯 Purpose:

- Creates a secure token after successful login
- Token contains user info (ID, role, email)
- Token expires after set time (24 hours)

How it's used:

```
python
```

```
# In auth.py when user logs in:
access_token = create_access_token(
    data={"sub": user_id, "role": user_role, "email": user_email}
)
return {"access_token": access_token, "token_type": "bearer"}
```

FILE 4: `scheduler.py` - Background Worker

This runs **periodic tasks** automatically:

```
python

import os
from datetime import datetime, timedelta
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from app.db.supabase_client import supabase
from app.api.v1.endpoints.whatsapp import send_whatsapp_message

scheduler = AsyncIOScheduler()
```

The Task: Follow-up on Leads

```
python

async def check_uncontacted_leads_and_followup():
    """Runs periodically to send re-engagement messages to pending leads."""
    print("⏰ Running 24-hour lead follow-up check...")
    try:
        # Find leads that haven't been contacted in 24 hours
        time_threshold = (datetime.utcnow() - timedelta(hours=24)).isoformat()

        res = supabase.table("leads")\
            .select("*")\
            .eq("status", "pending_contact")\
            .lte("created_at", time_threshold)\
            .execute()

        leads = getattr(res, "data", None) or []
        for lead in leads:
            phone = str(lead.get("customer_phone"))
```

```
            notes = lead.get("notes", "your inquiry")

            # Send WhatsApp follow-up
            followup_text = (
                f"Mambo! Karibu tena Ayutech Motors. "
                f"Tulipokea maombi yako kuhusu ({notes}). "
                f"Je, bado unahitaji usaidizi au ungetaka tukutumie part hii leo?"
            )

            await send_whatsapp_message(phone, followup_text)

            # Update lead status
            supabase.table("leads").update({"status": "followup_sent"}).eq("id", lead_i
d).execute()
```

Starting the Scheduler:

```
python

def start_scheduler():
    """Starts the background scheduler loop."""
    scheduler.add_job(check_uncontacted_leads_and_followup, 'interval', hours=12)
    scheduler.start()
    print("⏰ Background Lead Scheduler Active")
```

🎯 Purpose:

- Automatically checks for leads that haven't been contacted
- Sends follow-up WhatsApp messages every 12 hours
- Updates lead status to prevent duplicate messages

🔑 **KEY CONCEPT:** This runs **in the background** while your API is running. You don't have to manually trigger it!

FILE 5: `schemas.py` - Data Validation Models

This defines the **shape of data** for API requests/responses:

```
python
```

```
from pydantic import BaseModel
from typing import List, Optional

class OrderItemSchema(BaseModel):
    product_id: str
    quantity: int
    unit_price: float

class CheckoutRequest(BaseModel):
    customer_name: str
    phone_number: str
    shipping_address: str
    items: List[OrderItemSchema]
```

🎯 Purpose:

- Validates incoming data automatically
- Defines what fields are required
- Provides type checking

How it's used:

```
python

from app.schemas import CheckoutRequest

@router.post("/checkout")
async def checkout(payload: CheckoutRequest):  # Automatically validated!
    # payload.customer_name is a string
    # payload.phone_number is a string
    # payload.items is a list of OrderItemSchema
    # Each item has product_id, quantity, unit_price
```

🔑 **KEY CONCEPT:** If someone sends invalid data (e.g., `quantity: "abc"`), FastAPI automatically returns a 400 error before your code even runs!

🔄 HOW ALL CORE FILES WORK TOGETHER

text

```
┌─────────────────────────────────────────────────────────────────┐
│                    FASTAPI APPLICATION STARTUP                  │
│                                                                 │
│  1. Load config.py                                              │
│     → Reads .env file                                           │
│     → Exposes settings object                                   │
│                                                                 │
│  2. Initialize supabase_client.py                               │
│     → Uses settings.SUPABASE_URL                                │
│     → Uses settings.SUPABASE_KEY                                │
│     → Creates database connection                               │
│                                                                 │
│  3. Start scheduler.py                                          │
│     → Begins background tasks                                   │
│     → Runs every 12 hours                                       │
│                                                                 │
│  4. Register routes                                             │
│     → Each endpoint imports from:                               │
│       - config.py for settings                                  │
│       - supabase_client.py for DB                               │
│       - schemas.py for validation                               │
│       - rate_limiter.py for protection                          │
│       - security.py for auth                                    │
└─────────────────────────────────────────────────────────────────┘
```

📊 COMPLETE AUTHENTICATION FLOW

Here's how a user registers and logs in using all these files:

1. User Registers

python

```python
# auth.py
@router.post("/register")
async def register_user(user: UserRegister):
    # 1. Check if email exists in Supabase
    existing = supabase.table("users").select("id").eq("email", user.email).execute()
```

```
    # 2. Hash password using security.py
    hashed_pwd = get_password_hash(user.password)  # ← From security.py

    # 3. Save to database
    user_data = {
        "email": user.email,
        "password_hash": hashed_pwd,  # ← Stored as hash, not plain text!
        "full_name": user.full_name,
        "role": user.role
    }
    supabase.table("users").insert(user_data).execute()
```

2. User Logs In

python

```
# auth.py
@router.post("/token")
async def login(form_data: OAuth2PasswordRequestForm):
    # 1. Get user from database
    res = supabase.table("users").select("*").eq("email", form_data.username).execute()
    db_user = res.data[0]

    # 2. Verify password using security.py
    if not verify_password(form_data.password, db_user.get("password_hash")):
        raise HTTPException(status_code=400, detail="Incorrect password")

    # 3. Create JWT token using security.py
    access_token = create_access_token(
        data={"sub": db_user.get("id"), "role": db_user.get("role"), "email": db_user.g
et("email")}
    )

    return {"access_token": access_token, "token_type": "bearer"}
```

3. Protected Routes

python

```python
# Any protected route
@router.get("/profile")
async def get_profile(token: str = Depends(oauth2_scheme)):
    # 1. Decode token using security.py
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

    # 2. Verify user exists
    user_id = payload.get("sub")
    res = supabase.table("users").select("*").eq("id", user_id).execute()

    return res.data[0]
```

📝 QUIZ TIME!

Question 1: What's the purpose of `config.py`?

- A) To define database tables
- B) To centralize all configuration settings
- C) To handle M-Pesa payments
- D) To create the UI

Question 2: Why do we hash passwords instead of storing them as plain text?

- A) To save database space
- B) To make them unreadable if the database is compromised
- C) To make passwords longer
- D) To prevent users from forgetting passwords

Question 3: What does `@limiter.limit("5/minute")` do?

- A) Allows only 5 items per order
- B) Limits API requests to 5 per minute per IP
- C) Sets a 5% discount
- D) Limits the order value to KES 5,000

Question 4: What's the purpose of `scheduler.py` ?

- A) To schedule M-Pesa payments
- B) To run background tasks automatically
- C) To schedule staff shifts
- D) To plan product restocking

Question 5: In `security.py` , what does `create_access_token()` do?

- A) Generates a random password
- B) Creates a JWT token for authentication
- C) Creates a new user account
- D) Generates an M-Pesa STK push

Question 6: What happens if someone sends invalid data to an endpoint that uses Pydantic schemas?

- A) The data is ignored
- B) FastAPI automatically returns a 400 error
- C) The data is converted to the correct type
- D) The server crashes

Question 7: When does the scheduler run the lead follow-up task?

- A) Every hour
- B) Every 12 hours
- C) Only when manually triggered
- D) Once per day

Question 8: What's the purpose of `ACCESS_TOKEN_EXPIRE_MINUTES` in config.py?

- A) To set how long users stay logged in
- B) To set how long orders stay in the cart
- C) To set how long products stay in inventory
- D) To set how long WhatsApp messages are valid

Answer Key:

1. **B** - Centralizes all configuration settings
2. **B** - Makes them unreadable if the database is compromised
3. **B** - Limits API requests to 5 per minute per IP
4. **B** - Runs background tasks automatically
5. **B** - Creates a JWT token for authentication
6. **B** - FastAPI automatically returns a 400 error
7. **B** - Every 12 hours
8. **A** - Sets how long users stay logged in

🎯 COMPLETE SYSTEM UNDERSTANDING

You now understand **every single file** in the AyuTech Motors system:

Frontend Files (Flet)

- ✅ main_app.py - Customer app
- ✅ dashboard.py - Staff POS
- ✅ All views (home, browse, cart, orders, profile, product_detail, dashboard_view)

Backend Files (FastAPI)

- ✅ main.py - API entry point
- ✅ admin.py - Staff operations
- ✅ orders.py - Order processing
- ✅ payments.py - M-Pesa callbacks
- ✅ products.py - Catalog management
- ✅ ai.py - AI assistant
- ✅ whatsapp.py - WhatsApp webhooks

- ✅ auth.py - User authentication

Core Infrastructure

- ✅ supabase_client.py - Database connection
- ✅ config.py - Central configuration
- ✅ rate_limiter.py - Security protection
- ✅ security.py - Password & token handling
- ✅ scheduler.py - Background tasks
- ✅ schemas.py - Data validation

🚀 FINAL CHALLENGE: ADD A NEW FEATURE

Now that you understand the complete system, try adding this feature:

Feature: Order Cancellation by Staff

Step 1: Add a new endpoint in `admin.py`:

```
python

@router.patch("/orders/{order_id}/cancel")
async def cancel_order(order_id: str):
    # 1. Check if order is in "Pending" or "Paid" status
    # 2. If cancelled, restore stock (if it was deducted)
    # 3. Update status to "Cancelled"
    # 4. Send WhatsApp notification to customer
    return {"status": "success"}
```

Step 2: Add a "Cancel Order" button in `dashboard.py`:

```
python

if status in ["Pending PIN", "Paid"]:
    action_buttons.append(
        ft.ElevatedButton(
            "Cancel Order",
```

```
                bgcolor="#DC2626",
                color="white",
                on_click=lambda e, oid=db_id: cancel_order(oid)
            )
        )
```

Step 3: Add the frontend function:

```python
def cancel_order(order_id: str):
    try:
        res = httpx.patch(f"{API_BASE_URL}/admin/orders/{order_id}/cancel", timeout=4)
        if res.status_code == 200:
            show_toast("Order cancelled successfully!")
            refresh_all_data()
    except Exception as err:
        show_toast(f"Failed: {err}", is_error=True)
```

💡 FINAL THOUGHTS

You've now seen the **complete picture** of a real-world full-stack application:

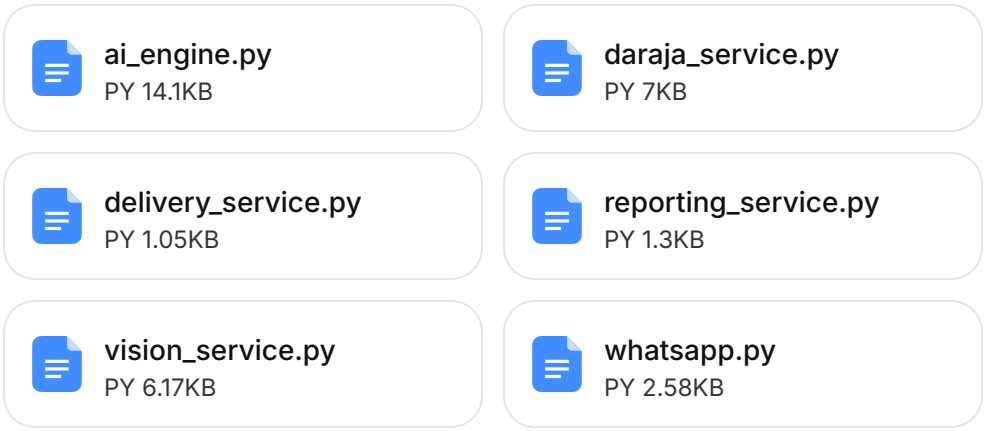
1. **Frontend:** Flet (Mobile + Desktop)
2. **Backend:** FastAPI (REST API)
3. **Database:** Supabase (PostgreSQL)
4. **External Services:** M-Pesa, WhatsApp, AI
5. **Security:** JWT tokens, password hashing, rate limiting
6. **Automation:** Background tasks (scheduler)

You've come a long way! 🎉

What would you like to explore next?

- 🔍 Deep dive into M-Pesa integration?
- 🤖 Understanding the AI engine?
- 🚀 Deployment to production?

- 🐛 Debugging common issues?
- 📱 Adding a new feature?



services

Excellent! You've shared the **services layer** - these are the specialized workers that handle specific tasks. This is where the **real magic** happens! Let me break down each service and show you how they power the entire system.

🏗️ SERVICES LAYER OVERVIEW

text

```
┌─────────────────────────────────────────────────────────────┐
│                      SERVICES LAYER                         │
│                                                             │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │ ai_engine.py │  │daraja_service│  │  whatsapp.py │       │
│  │  (AI Brain)  │  │   (M-Pesa)   │  │ (WhatsApp)   │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│                                                             │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │vision_service│  │delivery_     │  │reporting_    │       │
│  │  (Image AI)  │  │service.py    │  │service.py    │       │
│  │              │  │ (Delivery    │  │  (Reports)   │       │
│  │              │  │  Rates)      │  │              │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
└─────────────────────────────────────────────────────────────┘
```

📁 FILE-BY-FILE BREAKDOWN

FILE 1: `ai_engine.py` - The AI Brain

This is the **most complex** file - it handles all AI interactions:

A. System Setup

```
python

DELIVERY_RATES = {
    "machakos": 400,
    "thika": 300,
    "kiambu": 300,
    "nakuru": 600,
    "mombasa": 800,
    "eldoret": 700,
    "kisumu": 700,
    "nyeri": 400
}
```

🎯 **Purpose:** Defines delivery costs for different locations.

```
python

SYSTEM_INSTRUCTIONS = f"""
You are the official digital sales assistant for Ayutech Motors on Kirinyaga Road, Nair
obi.
Your job is to respond warmly, concisely, and professionally to WhatsApp inquiries.
Specialties: Toyota 1KD, 2KD, 3L, 5L, 7L, 9L engines, Hilux, HiAce, gearboxes, and serv
ice parts.

Response Rules:
1. When greeted informally (e.g. 'sasa', 'niaje', 'mambo'), reply naturally
2. Answer fitment, pricing, and stock inquiries in clear, direct English
3. Maximum 2 short sentences per reply
4. Reference exact inventory details from the list below
"""
```

🎯 **Purpose:** This is the **personality** of the AI - it tells the AI how to behave!

B. Core AI Functions

1. Check if Human Takeover is Active:

```
python

def is_human_takeover_active(customer_phone: str) -> bool:
    # Check if a human staff member has taken over this conversation
    res = supabase.table("leads").select("status").eq("customer_phone", customer_phon
e).execute()
    return status == "human_takeover"
```

🎯 **Purpose:** If a staff member takes over, the AI stops responding.

2. Query Inventory Database:

```
python

async def query_inventory_db(search_term: str) -> list:
    # Search for products matching the customer's query
    res = supabase.table("products").select("*").ilike("name", f"%{search_term}%").exec
ute()
    return res.data or []
```

🎯 **Purpose:** Find parts in inventory that match the customer's request.

3. Record Out-of-Stock Items:

```
python

def record_out_of_stock_wishlist(part_name: str):
    # Track what customers are asking for but we don't have
    supabase.table("stock_wishlist").insert({
        "part_name": clean_part,
        "request_count": 1,
        "last_requested_at": now_str
    }).execute()
```

🎯 **Purpose:** Track demand for parts that are out of stock - helps with restocking decisions!

4. Lead Qualification:

```python
def upsert_qualified_lead(customer_phone: str, notes: str, intent_level: str):
    # Save customer as a lead with intent level (cold/warm/hot)
    lead_data = {
        "business_id": "AYUTECH_MAIN",
        "customer_phone": customer_phone,
        "intent": intent_level,
        "notes": notes,
        "status": "pending_contact",
    }
    supabase.table("leads").insert(lead_data).execute()
```

🎯 **Purpose:** Track potential customers and their interest level.

5. Extract Context Entities:

```python
async def extract_context_entities(user_message: str, chat_history: list, groq_key: st
r) -> dict:
    # Use AI to understand the customer's intent
    extraction_prompt = f"""
    Analyze conversation and extract:
    {{
      "part_name": "the spare part discussed",
      "customer_wants_owner_contact": true/false,
      "intent": "cold, warm, or hot"
    }}
    """
    # Call Groq AI to analyze the conversation
```

🎯 **Purpose:** Understand what the customer wants and how interested they are.

6. Generate Reply:

```python
async def generate_grounded_reply(customer_phone: str, user_message: str, image_descrip
tion: str = "") -> str:
    # Check if human takeover is active
    if is_human_takeover_active(customer_phone):
        return ""
```

```
    # Get chat history
    chat_history = fetch_recent_chat_history(customer_phone, limit=6)

    # Extract context
    entities = await extract_context_entities(user_message, chat_history, groq_key)

    # Search inventory
    db_items = await query_inventory_db(part_name)

    # Build AI prompt with context
    messages_payload = [{"role": "system", "content": SYSTEM_INSTRUCTIONS}]

    # Add inventory context
    prompt_with_context = f"""
    DATABASE_CONTEXT: {json.dumps(db_items)}
    CUSTOMER_MESSAGE: {user_message}
    """

    # Call Groq AI
    res = await client.post(groq_endpoint, json=payload, headers=headers)
    reply = res.json()["choices"][0]["message"]["content"]

    return reply
```

🎯 **Purpose:** Generate a smart, context-aware reply to the customer.

C. Mobile App AI Assistant

python

```
async def generate_fitment_response(messages: list) -> dict:
    # Used by the mobile app's AI chat
    # 1. Search for matching products
    matched_products = []
    for p in all_products:
        if any(k in p_name or k in p_cat for k in keywords):
            matched_products.append(p)

    # 2. Generate AI response
    system_prompt = f"You are the AyuTech Motors Spare Parts Assistant..."

    # 3. Return response with matching products
```

```
    return {
        "reply": reply_text,
        "products": matched_products[:2]  # Top 2 matches for quick-add buttons
    }
```

🎯 **Purpose:** Helps customers find parts in the mobile app with instant "Add to Cart" buttons!

FILE 2: `daraja_service.py` - M-Pesa Integration

This handles **all M-Pesa communication**:

A. Get Access Token

```
python

@staticmethod
async def get_access_token() -> str:
    # Get OAuth token from Safaricom
    url = f"{BASE_URL}/oauth/v1/generate?grant_type=client_credentials"
    keys = f"{CONSUMER_KEY}:{CONSUMER_SECRET}"
    encoded_keys = base64.b64encode(keys.encode()).decode("utf-8")
    headers = {"Authorization": f"Basic {encoded_keys}"}

    res = await client.get(url, headers=headers)
    return res.json().get("access_token", "")
```

🎯 **Purpose:** Safaricom requires a token for every API call. This gets one.

B. Generate STK Push Password

```
python

@staticmethod
def generate_password(timestamp: str) -> str:
    # Create a secure password for the transaction
    raw_password = f"{BUSINESS_SHORT_CODE}{PASSKEY}{timestamp}"
    return base64.b64encode(raw_password.encode()).decode("utf-8")
```

🎯 **Purpose:** Safaricom requires a password for each transaction.

C. Initiate STK Push

```python
@classmethod
async def initiate_stk_push(cls, phone_number: str, amount: float, account_reference: s
tr, transaction_desc: str = "Payment") -> dict:
    # Send the payment prompt to customer's phone
    access_token = await cls.get_access_token()
    timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
    password = cls.generate_password(timestamp)

    # Format phone number
    formatted_phone = phone_number.strip().replace("+", "")
    if formatted_phone.startswith("0"):
        formatted_phone = f"254{formatted_phone[1:]}"

    # Build payload
    payload = {
        "BusinessShortCode": BUSINESS_SHORT_CODE,
        "Password": password,
        "Timestamp": timestamp,
        "TransactionType": "CustomerPayBillOnline",
        "Amount": int(amount),
        "PartyA": formatted_phone,
        "PartyB": BUSINESS_SHORT_CODE,
        "PhoneNumber": formatted_phone,
        "CallBackURL": CALLBACK_URL,
        "AccountReference": account_reference[:12],
        "TransactionDesc": transaction_desc[:30]
    }

    # Send to Safaricom
    res = await client.post(url, json=payload, headers=headers)
    return res.json()
```

🎯 **Purpose:** This is what sends the M-Pesa popup to the customer's phone!

🔑 **KEY CONCEPT:** The flow is:

1. Customer checks out → calls `initiate_stk_push()`

2. Safaricom sends popup to customer's phone
3. Customer enters PIN
4. Safaricom calls our `CALLBACK_URL` with the result

FILE 3: `whatsapp.py` - WhatsApp Notifications

This handles **all WhatsApp communication**:

python

```
WHATSAPP_TOKEN = os.getenv("WHATSAPP_TOKEN", "").strip()
WHATSAPP_PHONE_NUMBER_ID = os.getenv("WHATSAPP_PHONE_NUMBER_ID", "").strip()
OWNER_PHONE_NUMBER = os.getenv("OWNER_WHATSAPP_NUMBER", "254112323814").strip()
```

A. Send WhatsApp Message

python

```
async def send_whatsapp_message(to_phone: str, message: str) -> dict:
    # Send a text message via Meta WhatsApp Cloud API
    url = f"https://graph.facebook.com/v19.0/{WHATSAPP_PHONE_NUMBER_ID}/messages"
    headers = {
        "Authorization": f"Bearer {WHATSAPP_TOKEN}",
        "Content-Type": "application/json",
    }
    payload = {
        "messaging_product": "whatsapp",
        "to": target_phone,
        "type": "text",
        "text": {"body": message},
    }

    res = await client.post(url, json=payload, headers=headers)
    return res.json()
```

🎯 **Purpose:** Send any WhatsApp message to any number.

B. Send Order Alert

python

```python
async def send_order_whatsapp_alert(order_data: dict, receipt_number: str):
    # Format order summary and send to store owner
    items_summary = "\n".join([
        f"• {item.get('name')} (x{item.get('qty')}) - KES {price:,.0f}"
        for item in items
    ])

    message_text = f"""
    ✅ *NEW PAID ORDER - AYUTECH MOTORS*

    🧾 *Order ID:* {order_reference}
    💳 *M-Pesa Receipt:* {receipt_number}
    📞 *Customer Phone:* {customer_phone}
    💰 *Total Paid:* KES {total:,.0f}
    """

    return await send_whatsapp_message(OWNER_PHONE_NUMBER, message_text)
```

🎯 **Purpose:** Alert the store owner immediately when an order is paid.

FILE 4: `vision_service.py` - Image Recognition

This identifies **car parts from images**:

python

```python
async def describe_part_image(media_id_or_url: str, whatsapp_token: str, groq_api_key:
str = "") -> str:
    # 1. Download image from WhatsApp
    download_url = await get_whatsapp_media_url(media_id_or_url, whatsapp_token)
    img_res = await client.get(download_url, headers=headers)
    image_bytes = img_res.content

    # 2. Upload to Supabase Storage
    supabase.storage.from_("chat_image").upload(file=image_bytes, path=filename)

    # 3. Find an AI vision model
    active_model = await get_active_openrouter_vision_model(openrouter_key)
```

```
    # 4. Ask AI to identify the part
    prompt_text = """
    You are an auto spare parts specialist.
    Analyze this image and identify the exact automotive spare part.
    State the exact part name clearly.
    """

    payload = {
        "model": active_model,
        "messages": [
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": prompt_text},
                    {"type": "image_url", "image_url": {"url": data_url}}
                ]
            }
        ]
    }

    # 5. Return description
    return result["choices"][0]["message"]["content"]
```

🎯 **Purpose:** Customers can send a photo of a part, and the AI identifies it!

🔑 **KEY CONCEPT:** This uses **OpenRouter** which provides access to multiple AI vision models:

- Llama 3.2 Vision
- Other free vision models
- Dynamically picks the best available model

FILE 5: `delivery_service.py` - Delivery Rate Calculator

This calculates **delivery costs**:

```
python

DELIVERY_RATES = {
    "cbd": 200,
```

```
    "kirinyaga road": 0,
    "westlands": 300,
    "kilimani": 300,
    "upper hill": 300,
    "eastleigh": 250,
    "industrial area": 300,
    "parklands": 300,
    "thika road": 400,
    "kasarani": 400,
    "roysambu": 400,
    "ngong road": 350,
    "karen": 500,
    "ruaka": 450,
    "kitengela": 700,
    "ruiru": 600,
}

def calculate_delivery_fee(location_text: str) -> str:
    # Match location against rate matrix
    loc_lower = location_text.lower()
    for area, rate in DELIVERY_RATES.items():
        if area in loc_lower:
            if rate == 0:
                return "Free pickup available at Kirinyaga Road store!"
            return f"Delivery to {area.title()} is KES {rate}"

    return "Delivery within Nairobi ranges between KES 200 - KES 500"
```

🎯 **Purpose:** Tell customers how much delivery will cost based on their location.

FILE 6: `reporting_service.py` - Daily Reports

This generates **business reports**:

```
python

async def generate_daily_failure_report() -> dict:
    # Get today's start time
    today_start = datetime.utcnow().replace(hour=0, minute=0, second=0).isoformat()

    # Get top out-of-stock items customers requested
```

```
        wishlist_res = supabase.table("stock_wishlist")\
            .select("part_name, request_count")\
            .order("request_count", desc=True)\
            .limit(5)\
            .execute()

        # Get failed vision attempts
        failed_vision = supabase.table("system_logs")\
            .select("*")\
            .eq("event_type", "VISION_FAILURE")\
            .gte("created_at", today_start)\
            .execute()

        report = {
            "date": str(datetime.utcnow().date()),
            "top_out_of_stock_restock_items": wishlist_res.data,
            "total_vision_failures": len(failed_vision.data),
            "health_status": "HEALTHY"
        }

        return report
```

🎯 **Purpose:** Generate daily reports about:

- Which out-of-stock parts customers want most
- How many vision failures occurred
- Overall system health

🔄 HOW SERVICES WORK TOGETHER: CUSTOMER WHATSAPP FLOW

Let me trace a complete customer interaction via WhatsApp:

1. Customer Sends Message

```
text

Customer: "Niaje! Nataka brake pads za Hilux"
```

(Translation: "Hi! I want brake pads for Hilux")

2. WhatsApp Webhook Receives

text

whatsapp.py (webhook) → ai_engine.py → generate_grounded_reply()

3. AI Processes

python

```python
# ai_engine.py
def generate_grounded_reply(customer_phone, user_message):
    # 1. Check if human takeover is active
    if is_human_takeover_active(phone):
        return ""  # AI is silent, human handles it

    # 2. Get chat history
    history = fetch_recent_chat_history(phone, limit=6)

    # 3. Extract context
    entities = extract_context_entities(user_message, history, groq_key)
    # entities = {"part_name": "brake pads", "intent": "hot"}

    # 4. Search inventory
    db_items = query_inventory_db("brake pads")
    # Finds all brake pads in stock

    # 5. Build AI prompt with inventory context
    prompt = f"""
    DATABASE_CONTEXT: {db_items}
    CUSTOMER_MESSAGE: {user_message}
    """

    # 6. Generate response
    reply = groq_ai.generate(prompt)
    # Reply: "Tuko na brake pads za Hilux. Bei ni KES 3,500. Ziko stock."

    # 7. Save to conversation history
    save_chat_message(phone, "assistant", reply)
```

```
    # 8. Send WhatsApp reply
    send_whatsapp_message(phone, reply)
```

4. If Customer Shows Interest

```
text

Customer: "Sawa, nitanunua. Niunganishe na owner."
(Translation: "OK, I'll buy. Connect me with the owner.")
```

5. AI Triggers Lead Capture

```
python

# ai_engine.py
entities = extract_context_entities(user_message, history, groq_key)
# entities = {"customer_wants_owner_contact": True, "intent": "hot"}

# Save as hot lead
upsert_qualified_lead(phone, "Brake pads for Hilux, wants to buy", "hot")
# → Lead saved to Supabase with status "pending_contact"
# → Staff dashboard shows this lead with "HOT" badge

# Reply
reply = "Nimemjulisha owner wa Ayutech Motors. Akutafutia hii part na atakujibu hivi ka
ribuni!"
```

6. Staff Sees Lead in Dashboard

```
text

Staff Dashboard (dashboard.py) → Leads Tab
  → Shows "📞 2547XXXXXXX (HOT)" with "Takeover Lead" button
  → Staff clicks "Takeover Lead"
  → Updates lead status to "human_takeover"
  → AI stops responding, staff handles conversation
```

📝 QUIZ TIME!

Question 1: What does the `ai_engine.py` do when a customer asks for a part that's out of stock?

- A) Ignores the request
- B) Records it in the stock wishlist for future restocking
- C) Automatically orders it from the supplier
- D) Deletes the part from the database

Question 2: What does `daraja_service.py` do with the phone number before sending an STK push?

- A) Converts it to 254 format
- B) Deletes it
- C) Sends it to WhatsApp
- D) Saves it to the database

Question 3: When does `whatsapp.py` send an order alert to the owner?

- A) When an order is created
- B) When payment is successful
- C) When the order is fulfilled
- D) When the customer cancels

Question 4: What does `vision_service.py` use to identify car parts from images?

- A) OCR (text recognition)
- B) AI vision models (like Llama 3.2 Vision)
- C) Barcode scanning
- D) Manual human review

Question 5: What's the purpose of `delivery_service.py` ?

- A) To calculate delivery fees based on location
- B) To track delivery drivers

- C) To generate delivery receipts
- D) To schedule deliveries

Question 6: In `ai_engine.py`, what does `record_out_of_stock_wishlist()` do?

- A) Sends a complaint to the supplier
- B) Tracks what customers are asking for but we don't have
- C) Automatically restocks the item
- D) Deletes the item from inventory

Question 7: What happens when `is_human_takeover_active()` returns True?

- A) The AI responds with a special message
- B) The AI stops responding
- C) The AI escalates to a supervisor
- D) The AI sends a follow-up message

Question 8: In `daraja_service.py`, what is the `CALLBACK_URL` used for?

- A) To send the STK push
- B) To receive payment results from Safaricom
- C) To check order status
- D) To send WhatsApp messages

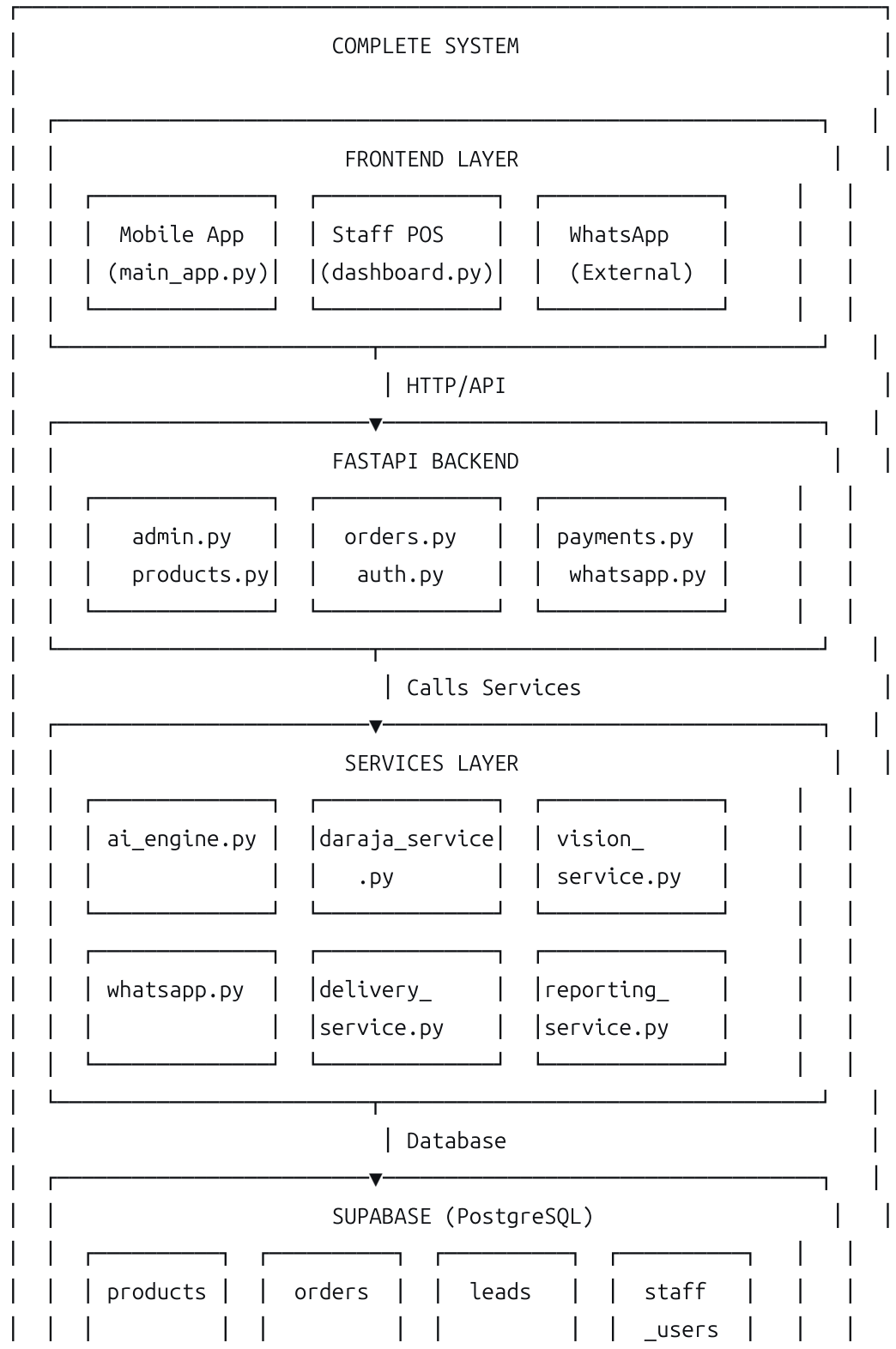
Answer Key:

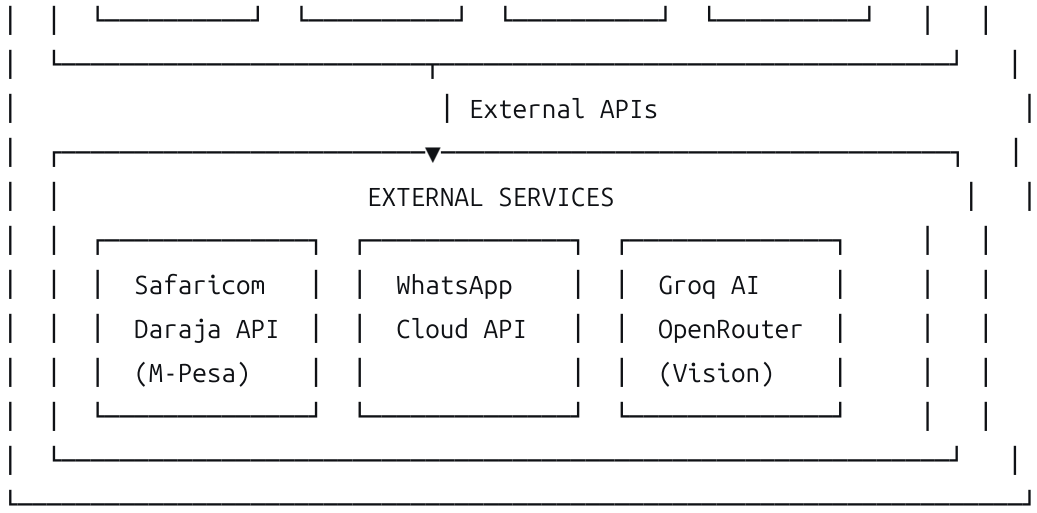
1. **B** - Records it in the stock wishlist for future restocking
2. **A** - Converts to 254 format (Kenyan standard)
3. **B** - When payment is successful
4. **B** - AI vision models (like Llama 3.2 Vision)
5. **A** - To calculate delivery fees based on location
6. **B** - Tracks what customers are asking for but we don't have
7. **B** - The AI stops responding (human takeover)

8. **B** - To receive payment results from Safaricom

🎯 COMPLETE SYSTEM ARCHITECTURE SUMMARY

text





💡 KEY TAKEAWAYS

1. **Services are specialized workers** - Each does one thing well
2. **AI Engine is the brain** - Understands customer intent and generates responses
3. **Daraja Service handles M-Pesa** - All payment logic in one place
4. **WhatsApp Service sends notifications** - Order alerts, receipts, follow-ups
5. **Vision Service identifies parts** - AI-powered image recognition
6. **Services are reusable** - Can be called from any endpoint

🚀 FINAL CHALLENGE

Now that you understand the entire system, try adding this feature:

Feature: Automated Stock Alert

When stock reaches 0, automatically send a WhatsApp alert to the owner:

1. **In** `admin.py` after stock update:

```
python
```

```
if new_stock == 0:
    await send_whatsapp_message(OWNER_PHONE, f"⚠️ {p_name} is OUT OF STOCK!")
```

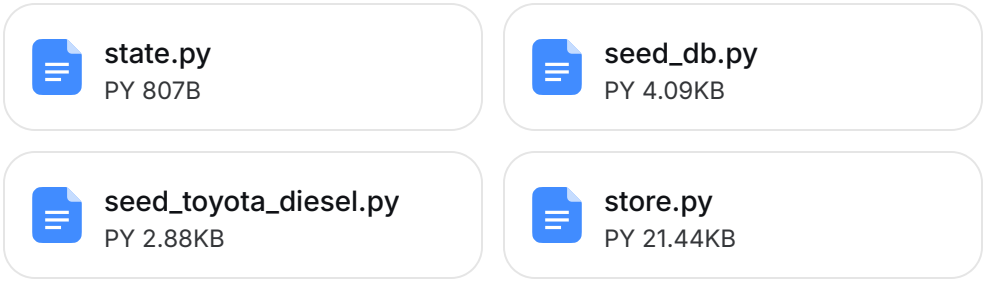
2. **In** `ai_engine.py` when a customer asks for out-of-stock:

```
python

if not db_items:
    await send_whatsapp_message(OWNER_PHONE, f"📞 Customer asked for {part_name} but w
e're out of stock!")
```

🎉 **CONGRATULATIONS!** You've now learned the **ENTIRE** AyuTech Motors system from frontend to backend to services to database!

You're ready to build, modify, and deploy this system yourself! 🚀



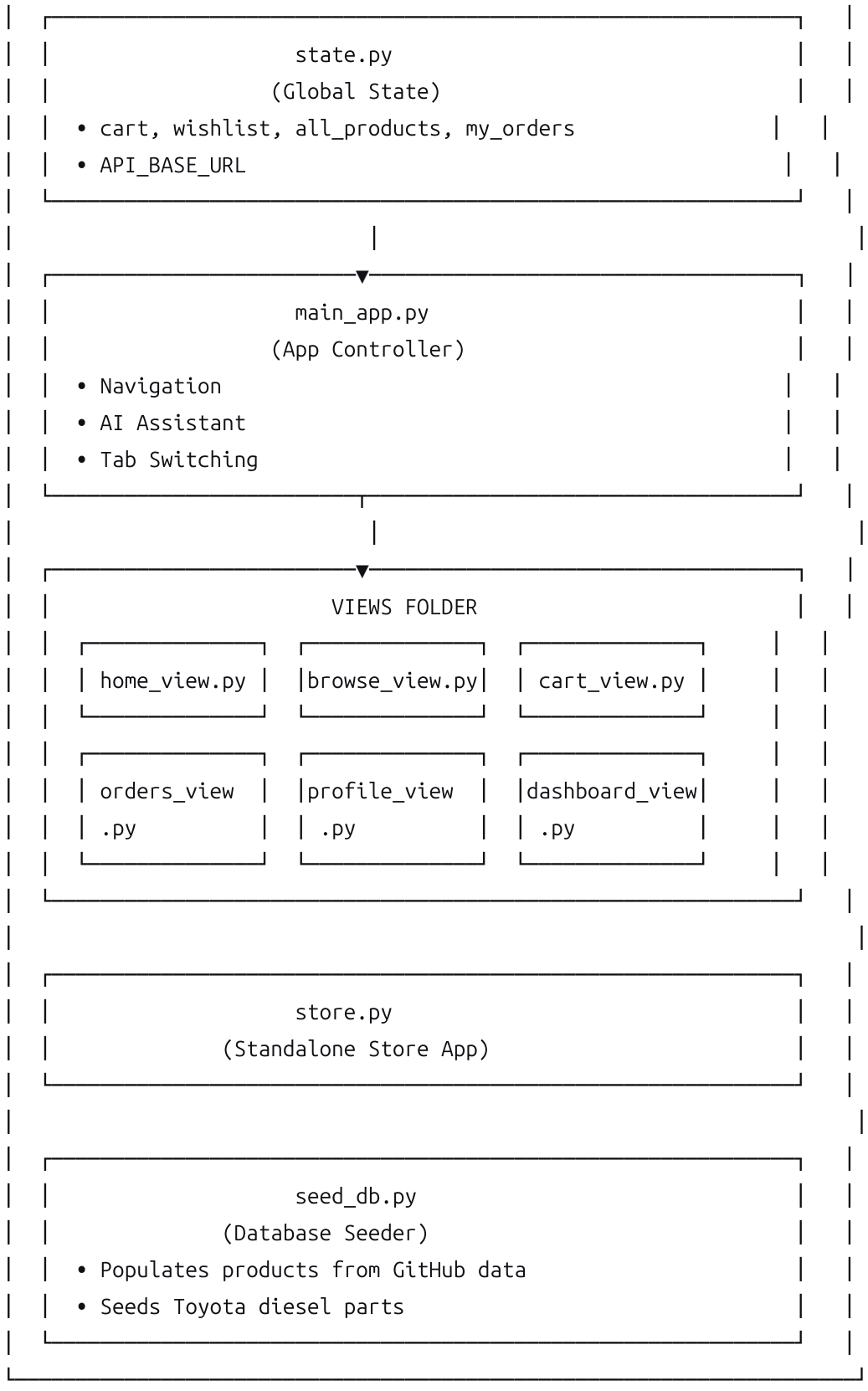
ui folder

Excellent! You've shared the **UI folder** - the customer-facing application layer. Now I can show you the **complete picture** of the customer mobile app and its supporting files. Let me break down each file and show you how they all work together!

🏗️ UI FOLDER STRUCTURE

```
text

┌──────────────────────────────────────────────────────────────────────────┐
│                          UI  FOLDER                                      │
│                                                                          │
```



📁 FILE-BY-FILE BREAKDOWN

FILE 1: `state.py` - Global Application State

This is the **central data store** for the mobile app:

```python
import flet as ft

# Global App State
cart = {}      # {id: {name, price, qty, image}}
wishlist = {}  # {id: {name, price, image}}
all_products = []
current_category = "All"
my_orders = []
```

🎯 Purpose:

- Stores data that needs to be accessed across multiple views
- Singleton pattern - one source of truth
- Persists data between navigation switches

Sample Data:

```python
all_products = [
    {"id": "1", "name": "5-Speed Gearbox Assembly", "price": 65000, "category": "Gear P
arts"},
    {"id": "2", "name": "Air Cleaner Housing 1KD Diesel", "price": 4500, "category": "E
ngine Parts"},
]
```

API Configuration:

```python
API_BASE_URL = "http://127.0.0.1:8000/api/v1"
```

🎯 **Purpose:** The base URL for all backend API calls.

🔑 **KEY CONCEPT:** By importing from `state.py`, all views share the same data:

```
python

from app.ui.state import cart, wishlist, all_products, API_BASE_URL
# Now any view can access and modify these global variables
```

FILE 2: `store.py` - Standalone Store App

This is a **self-contained version** of the customer app:

Key Features:

1. **Dark/Light Theme Support**
2. **Product Browsing & Search**
3. **Cart Management**
4. **Wishlist**
5. **Checkout with M-Pesa/Cash**
6. **AI Assistant Modal**

Theme Toggle:

```
python

def toggle_theme(e):
    if page.theme_mode == ft.ThemeMode.DARK:
        page.theme_mode = ft.ThemeMode.LIGHT
        page.bgcolor = "#F8F9FA"
    else:
        page.theme_mode = ft.ThemeMode.DARK
        page.bgcolor = "#0B0B0B"
    render_home()
    render_browse(current_category)
    page.update()
```

Product Card:

```
python
```

```
def build_product_card(p):
    # Creates a consistent product card with:
    # - Image (with fallback)
    # - Name
    # - Price
    # - Wishlist button
    # - Add to Cart button
    return ft.Container(
        bgcolor=card_bg, border_radius=12, padding=10,
        content=ft.Column([
            ft.Stack([
                ft.Container(height=90, content=create_product_image(p.get("image_ur
l"))),
                ft.IconButton(..., on_click=lambda e, item=p: toggle_wishlist(item))
            ]),
            ft.Text(p["name"], size=12, weight=ft.FontWeight.BOLD, max_lines=2),
            ft.Row([
                ft.Text(f"KES {p['price']:,.0f}", size=12, color="#DC2626"),
                ft.IconButton(..., on_click=lambda e, item=p: add_to_cart(item))
            ])
        ])
    )
```

Cart View:

```
python

def render_cart_page():
    # 1. Display all cart items
    for p_id, item in cart.items():
        items_col.controls.append(
            ft.Container(
                content=ft.Row([
                    ft.Container(width=55, height=55, content=create_product_image(item
["image"])),
                    ft.Column([
                        ft.Text(item["name"], size=12, weight=ft.FontWeight.BOLD),
                        ft.Text(f"KES {item['price']:,.0f}", size=12, color="#DC2626")
                    ]),
                    ft.Row([
                        ft.IconButton(..., on_click=lambda e: update_cart_qty(p_id, -
1)),
                        ft.Text(str(item["qty"])),
```

```
                            ft.IconButton(..., on_click=lambda e: update_cart_qty(p_id,
1)),
                        ])
                    ])
                )
            )

        # 2. Show checkout form
        checkout_box = ft.Column([
            ft.Text("Select Payment Method:"),
            payment_method,  # M-Pesa / Cash radio buttons
            phone_input,     # Phone number for M-Pesa
            checkout_msg,    # Status message
            ft.ElevatedButton("Complete Order", on_click=trigger_checkout)
        ])
```

Checkout:

python

```
def trigger_checkout(e):
    # 1. Validate phone number (if M-Pesa)
    if mode == "mpesa" and not phone:
        checkout_msg.value = "⚠️ Please enter your M-Pesa phone number!"
        return

    # 2. Send order to backend
    res = httpx.post(f"{API_BASE_URL}/payments/orders/create", json={
        "customer_phone": phone or "CASH_CUSTOMER",
        "items": items_list,
        "total_amount": total_amt
    })

    # 3. Trigger STK Push if M-Pesa
    if mode == "mpesa":
        stk = httpx.post(f"{API_BASE_URL}/payments/stk-push", json={
            "phone_number": phone, "amount": 1,
            "callback_url": "https://.../mpesa-callback",
            "checkout_request_id": order_id
        })
        checkout_msg.value = "📲 M-Pesa Prompt sent! Check your phone."
```

FILE 3: `seed_db.py` - Database Seeder

This populates the database with **initial product data**:

How It Works:

1. **Fetches data from GitHub:**

```python
GITHUB_RAW_BASE = "https://raw.githubusercontent.com/wachiraericksonkinyua/ayutech/main/"

data_files = {
    "bodyparts.js": "Body Parts",
    "brakeparts.js": "Brake Parts",
    "engineparts.js": "Engine Parts",
    "gearparts.js": "Gear Parts",
    "lubricants.js": "Lubricants",
    "serviceparts.js": "Service Parts",
    "suspensionparts.js": "Suspension Parts"
}
```

2. **Parses JavaScript arrays:**

```python
def parse_js_array_to_list(js_content: str) -> list:
    # Converts JS array like: [{id: 1, name: "Part"}, ...]
    # To Python list: [{"id": 1, "name": "Part"}, ...]

    # 1. Clean non-breaking spaces
    clean_content = js_content.replace('\xa0', ' ')

    # 2. Extract array content inside [...]
    match = re.search(r'\[.*\]', clean_content, re.DOTALL)

    # 3. Strip comments
    array_str = re.sub(r'//.*?\n', '\n', array_str)
    array_str = re.sub(r'/\*.*?\*/', '', array_str, re.DOTALL)

    # 4. Quote unquoted keys (id: → "id":)
    array_str = re.sub(r'(\b[a-zA-Z_]\w*\b)\s*:', r'"\1":', array_str)
```

```
    # 5. Convert JS booleans to Python
    array_str = re.sub(r'\btrue\b', 'True', array_str)
    array_str = re.sub(r'\bfalse\b', 'False', array_str)
    array_str = re.sub(r'\bnull\b', 'None', array_str)

    # 6. Remove trailing commas
    array_str = re.sub(r',\s*([\]}])', r'\1', array_str)

    return ast.literal_eval(array_str)
```

3. **Inserts into Supabase:**

```
python

def seed_all_categories_from_github():
    for filename, category_name in data_files.items():
        # 1. Download from GitHub
        res = client.get(raw_url)

        # 2. Parse items
        items = parse_js_array_to_list(res.text)

        # 3. Insert each item
        for item in items:
            payload = {
                "name": p_name,
                "category": category_name,
                "price": price,
                "stock_quantity": 10,
                "image_url": img_url
            }
            supabase.table("products").insert(payload).execute()
```

🎯 Purpose:

- Populate the database with hundreds of products
- Ensures the catalog is populated without manual entry
- Can be run anytime to reset or update the catalog

FILE 4: `seed_toyota_diesel.py` - Toyota Diesel Parts Seeder

This seeds **specific Toyota diesel parts**:

```
python

toyota_diesel_parts = [
    # 1KD / 2KD D4D Components
    {
        "name": "Toyota 1KD Complete Cylinder Head (Hilux/Prado)",
        "category": "Engine Components",
        "price": 48000.0,
        "stock_quantity": 4,
    },
    {
        "name": "Toyota 2KD D4D Turbocharger CT16 (HiAce/Hilux)",
        "category": "Turbo & Induction",
        "price": 28500.0,
        "stock_quantity": 5,
    },
    # ... more parts
]
```

🎯 Purpose:

- Adds specialized diesel engine parts
- Complements the general catalog
- Focuses on AyuTech's core specialty (Toyota diesel engines)

🔄 COMPLETE DATA FLOW: UI TO BACKEND

Let me trace a **complete customer interaction** through the UI:

1. App Starts

```
text

store.py → main()
  → fetch_data()
```

```
  → GET /api/v1/admin/products
  → Backend returns product list
  → all_products = res.json()
  → render_home() / render_browse()
```

2. Customer Browses Products

```
text

store.py → render_browse()
  → Filter products by category
  → Build product cards with build_product_card()
  → Display in GridView
```

3. Customer Adds to Cart

```
text

store.py → add_to_cart(p)
  → Add to cart dictionary: cart[p_id] = {...}
  → Update UI with SnackBar
```

4. Customer Views Cart

```
text

store.py → switch_tab_index(2)
  → render_cart_page()
  → Display all items in cart
  → Show total and checkout form
```

5. Customer Checks Out

```
text

store.py → trigger_checkout(e)
  → POST /api/v1/payments/orders/create
  → Backend creates order
```

```
  → If M-Pesa: POST /payments/stk-push
  → Backend sends STK push
  → Customer enters PIN on phone
  → Backend receives callback
  → Order status updates to "Paid"
```

6. Customer Sees Order Status

```
text

store.py → Orders tab (when implemented)
  → GET /api/v1/orders/status/{order_id}
  → Shows "Paid" status
```

📊 COMPLETE FILE STRUCTURE SUMMARY

```
text

backend/
├── app/
│   ├── api/v1/endpoints/
│   │   ├── admin.py          # Staff operations
│   │   ├── orders.py         # Order processing
│   │   ├── payments.py       # M-Pesa callbacks
│   │   ├── products.py       # Product catalog
│   │   ├── ai.py            # AI assistant
│   │   ├── whatsapp.py      # WhatsApp webhooks
│   │   └── auth.py          # User authentication
│   │
│   ├── services/
│   │   ├── ai_engine.py     # AI brain
│   │   ├── daraja_service.py # M-Pesa
│   │   ├── whatsapp.py      # WhatsApp
│   │   ├── vision_service.py # Image recognition
│   │   ├── delivery_service.py # Delivery rates
│   │   └── reporting_service.py # Reports
│   │
│   ├── core/
│   │   ├── config.py        # Settings
│   │   ├── security.py      # Password/JWT
```

```
│   │   ├── rate_limiter.py  # Protection
│   │   └── scheduler.py     # Background tasks
│   │
│   ├── db/
│   │   └── supabase_client.py # Database connection
│   │
│   └── ui/
│       ├── state.py         # Global state
│       ├── main_app.py      # App controller
│       ├── store.py         # Standalone app
│       ├── views/
│       │   ├── home_view.py
│       │   ├── browse_view.py
│       │   ├── cart_view.py
│       │   ├── orders_view.py
│       │   ├── profile_view.py
│       │   ├── dashboard_view.py
│       │   └── product_detail_view.py
│       │
│       └── seed_db.py       # Database seeder
```

📝 QUIZ TIME!

Question 1: What is the purpose of `state.py`?

- A) To store UI theme settings
- B) To hold global application state (cart, wishlist, products)
- C) To connect to the database
- D) To handle M-Pesa payments

Question 2: What does `seed_db.py` do?

- A) Seeds the database with products from GitHub
- B) Creates database tables
- C) Deletes all products
- D) Updates product prices

Question 3: In `store.py`, what happens when a customer adds a product to the cart?

- A) It's immediately purchased
- B) It's added to the cart dictionary
- C) It's saved to the database
- D) It's sent to WhatsApp

Question 4: How does `store.py` handle theme switching?

- A) By reloading the entire app
- B) By changing `page.theme_mode` and re-rendering
- C) By using CSS styles
- D) By fetching new data

Question 5: What is `API_BASE_URL` used for?

- A) To connect to the Supabase database
- B) To define the backend API endpoint
- C) To set the frontend URL
- D) To configure M-Pesa

Question 6: In `seed_db.py`, why does it parse JavaScript arrays?

- A) To support JavaScript frontends
- B) Because the data on GitHub is in JavaScript format
- C) To convert JSON to JavaScript
- D) To validate the data

Question 7: What does `seed_toyota_diesel.py` add to the database?

- A) General spare parts
- B) Toyota diesel engine specific parts
- C) M-Pesa transaction records
- D) Staff user accounts

Question 8: In `store.py`, what happens during checkout with M-Pesa?

- A) The customer pays cash

- B) An STK push is sent to the customer's phone
- C) The order is cancelled
- D) The customer receives a refund

Answer Key:

1. **B** - Holds global application state
2. **A** - Seeds the database with products from GitHub
3. **B** - Added to the cart dictionary
4. **B** - Changes `page.theme_mode` and re-renders
5. **B** - Defines the backend API endpoint
6. **B** - The data on GitHub is in JavaScript format
7. **B** - Toyota diesel engine specific parts
8. **B** - An STK push is sent to the customer's phone

🎯 COMPLETE UNDERSTANDING CHECK

You now understand the **ENTIRE AyuTech Motors system**:

Frontend Layer

- ✅ **Mobile App** (`main_app.py` , `store.py`)
- ✅ **Staff POS** (`dashboard.py`)
- ✅ **All Views** (home, browse, cart, orders, profile, product_detail)
- ✅ **Global State** (`state.py`)
- ✅ **Database Seeding** (`seed_db.py`)

Backend Layer

- ✅ **API Endpoints** (admin, orders, payments, products, ai, whatsapp, auth)

- ✅ **Services** (ai_engine, daraja, whatsapp, vision, delivery, reporting)
- ✅ **Core Infrastructure** (config, security, rate_limiter, scheduler)
- ✅ **Database** (supabase_client)

Data Flow

- ✅ **User Interface → API → Services → Database**
- ✅ **Database → API → User Interface**
- ✅ **External APIs** (M-Pesa, WhatsApp, Groq AI, OpenRouter)

Business Logic

- ✅ **Order Processing** (checkout → payment → fulfillment)
- ✅ **Inventory Management** (stock tracking, refill requests)
- ✅ **Customer Engagement** (AI chat, leads, WhatsApp notifications)
- ✅ **Sales Reporting** (analytics, daily reports, EOD summaries)

🚀 FINAL CHALLENGE

Now that you understand everything, try adding a new feature:

Feature: Product Reviews

1. **Database:** Add `reviews` table with:
   - `id`, `product_id`, `customer_name`, `rating` (1-5), `comment`, `created_at`
2. **Backend:** Add endpoints in `products.py`:

```python
@router.post("/{product_id}/reviews")
async def add_review(product_id: str, payload: ReviewPayload):
    # Save review to database
    return {"status": "success"}
```

```
@router.get("/{product_id}/reviews")
async def get_reviews(product_id: str):
    # Fetch reviews for a product
    return reviews
```

3. **UI:** Add a "Reviews" section in `product_detail_view.py`:

```
python

# Display stars based on rating
ft.Row([
    ft.Icon(ft.icons.STAR, color="#F59E0B") for _ in range(rating)
])

# Add review form
ft.TextField(label="Your Name")
ft.Dropdown(label="Rating", options=[1,2,3,4,5])
ft.TextField(label="Comment", multiline=True)
ft.ElevatedButton("Submit Review")
```

🎉 **CONGRATULATIONS!** You've completed the **ENTIRE AyuTech Motors system**!

You now have a **comprehensive understanding** of:

- 🖥️ **Full-stack development** (Frontend + Backend)
- 💰 **Payment integration** (M-Pesa)
- 🤖 **AI integration** (Chat, Vision)
- 💬 **Messaging** (WhatsApp)
- 🗄️ **Database** (Supabase/PostgreSQL)
- 🔐 **Security** (JWT, Rate Limiting)
- 📊 **Reporting** (Analytics)

You're ready to build, maintain, and extend this system! 🚀

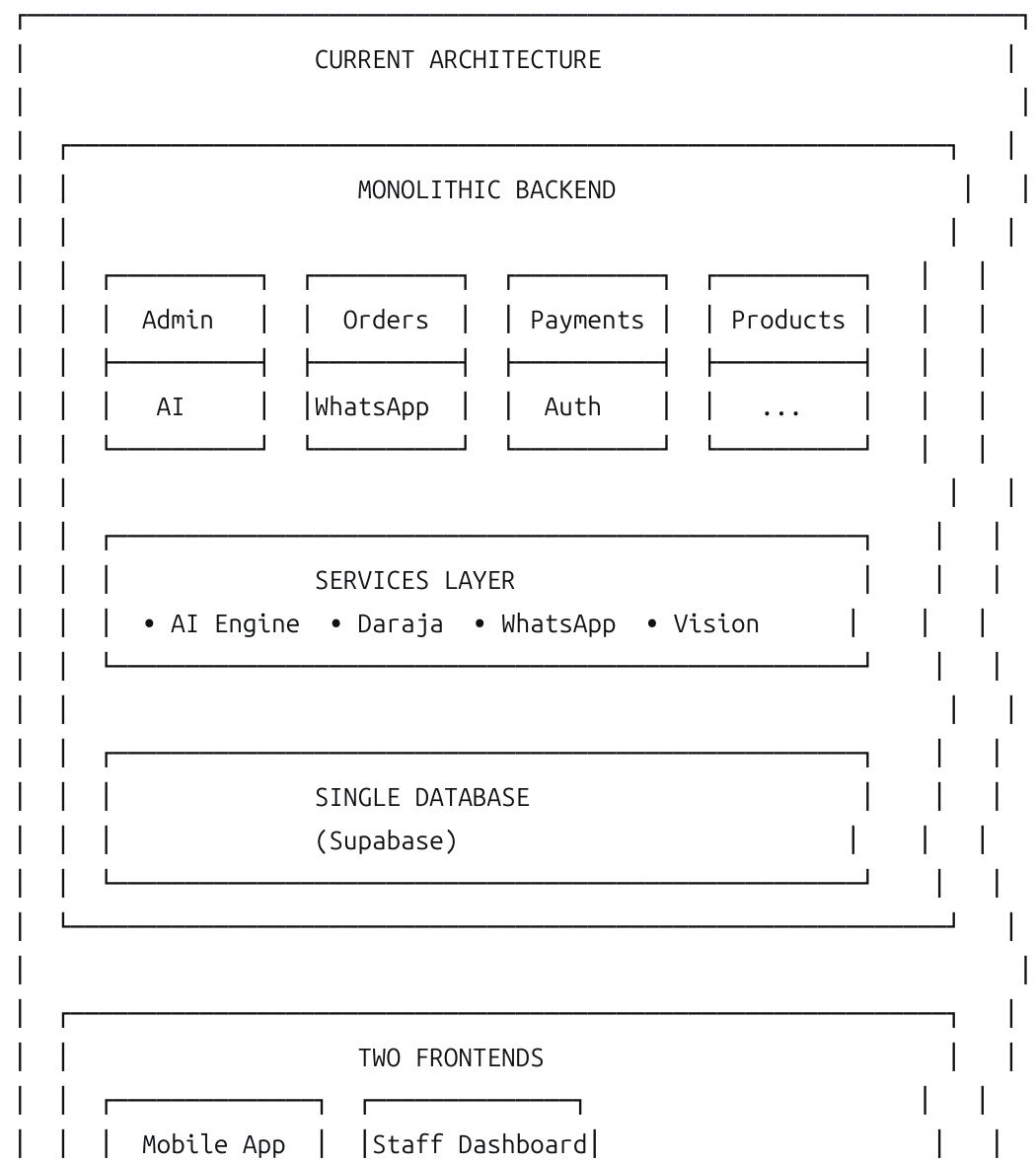
whts next? do you need how do they connect some architectures you advice me to use, what i should i know

Excellent question! You've learned the **entire codebase** - now let me show you the **architecture patterns** and **best practices** that make this system work. This is the **next level** - understanding *why* things are structured the way they are and *how* to improve them.

🏗️ CURRENT ARCHITECTURE VS. RECOMMENDED ARCHITECTURE

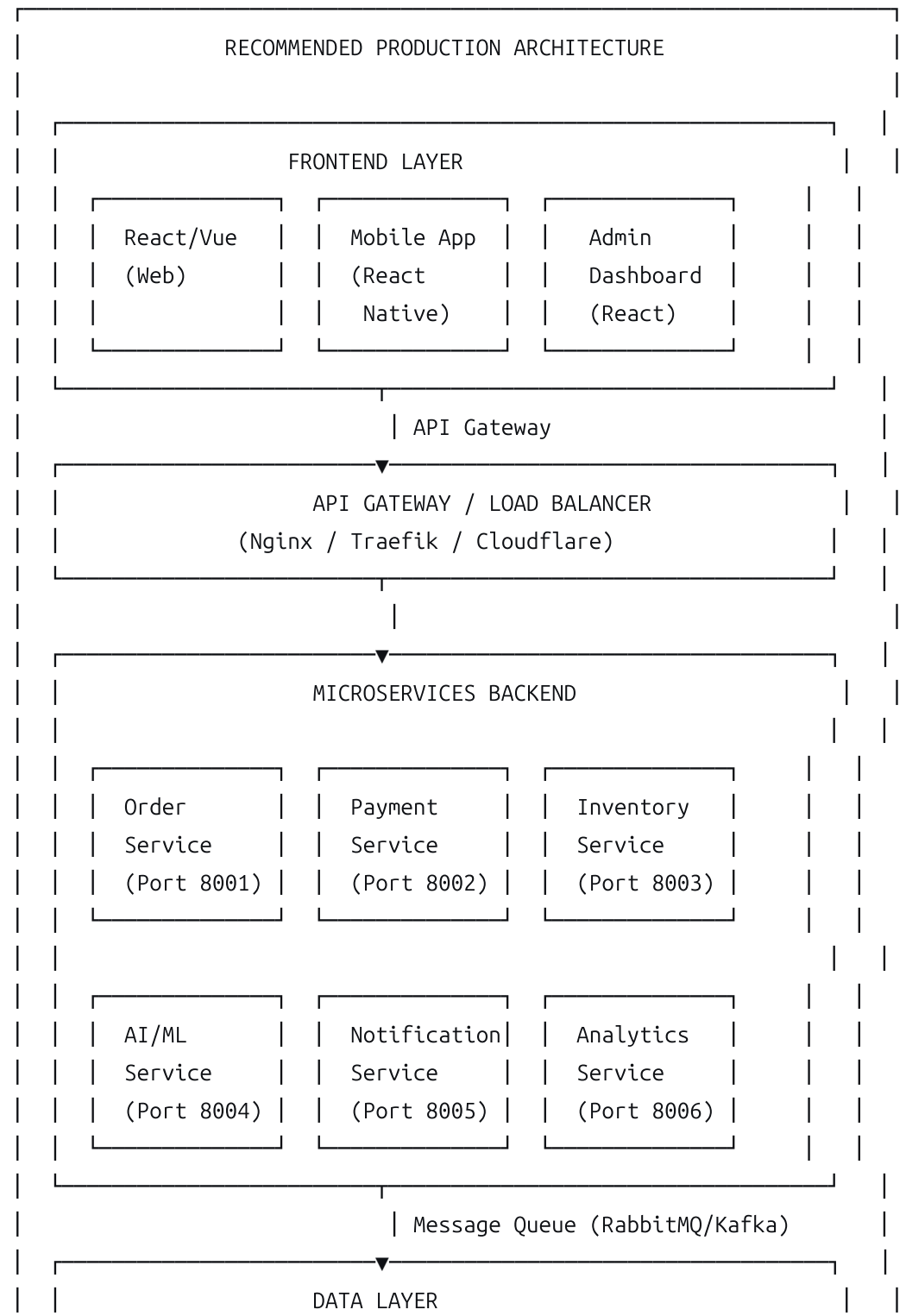
What You Have Now (Current Architecture)

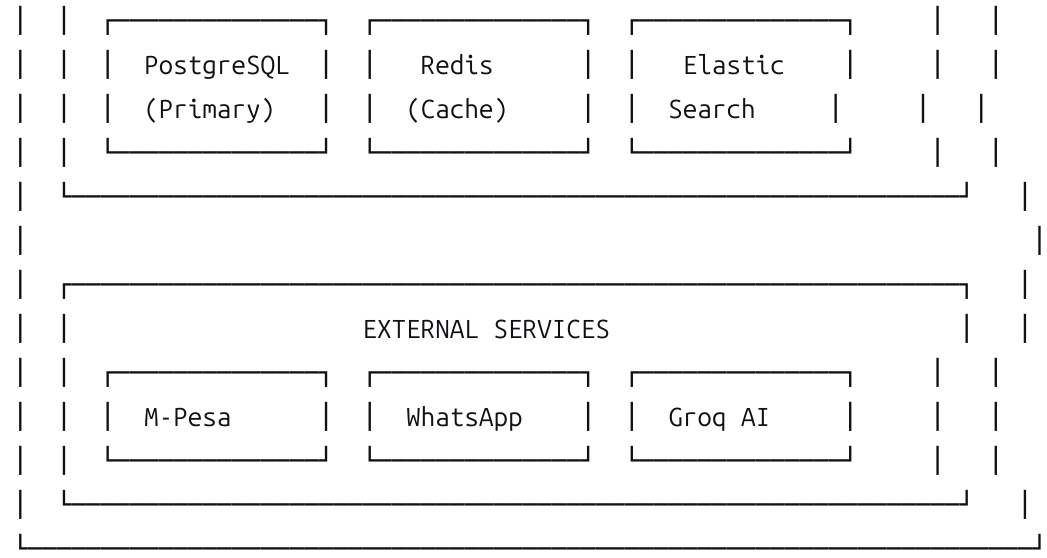
text



Recommended Architecture (For Production)

text





🎯 WHAT YOU SHOULD KNOW NEXT

1. DEPLOYMENT ARCHITECTURE

Current Setup (Local Development)

```yaml
# Everything runs on localhost
Frontend: http://localhost:8000 (Flet)
Backend: http://localhost:8000 (FastAPI)
Database: Supabase Cloud
```

Production Setup (Recommended)

```yaml
# Option A: Single Server (Small Scale)
Server:
  - Ubuntu 22.04
  - Nginx (Reverse Proxy)
  - Supervisor (Process Management)
  - Gunicorn (WSGI Server)
  - SSL Certificate (Let's Encrypt)

Services:
```

```
  Backend: http://api.ayutech.co.ke
  Frontend: http://ayutech.co.ke
  Database: Supabase Cloud

# Option B: Docker Compose (Medium Scale)
docker-compose.yml:
  services:
    backend:
      build: ./backend
      ports: ["8000:8000"]
      environment:
        - SUPABASE_URL=...
        - SUPABASE_KEY=...

    nginx:
      image: nginx:alpine
      ports: ["80:80", "443:443"]
      volumes:
        - ./nginx.conf:/etc/nginx/conf.d/default.conf

    redis:
      image: redis:alpine
      ports: ["6379:6379"]

    postgres:
      image: postgres:15
      volumes:
        - ./postgres_data:/var/lib/postgresql/data

# Option C: Cloud (Large Scale)
Cloud Providers:
  - AWS (EC2, RDS, ElastiCache)
  - Google Cloud (Compute Engine, Cloud SQL)
  - Azure (Virtual Machines, Azure SQL)
  - Heroku / Railway / Render (PaaS)
```

2. DATABASE OPTIMIZATION

Current Issues

python

```python
# × Current: No indexes
orders_query = supabase.table("orders").select("*").order("created_at", desc=True)

# × Current: No caching
products = supabase.table("products").select("*").execute()
```

Recommended Optimizations

A. Database Indexes:

```sql
-- Supabase SQL Editor
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_created_at ON orders(created_at DESC);
CREATE INDEX idx_orders_customer_phone ON orders(customer_phone);
CREATE INDEX idx_products_category ON products(category);
CREATE INDEX idx_products_name ON products(name);
```

B. Caching Strategy:

```python
# Add Redis caching
import redis
import json

redis_client = redis.Redis(host='localhost', port=6379, decode_responses=True)

async def get_products_cached():
    # Try cache first
    cached = redis_client.get("products:all")
    if cached:
        return json.loads(cached)

    # Fetch from database
    products = supabase.table("products").select("*").execute()

    # Cache for 5 minutes
    redis_client.setex("products:all", 300, json.dumps(products.data))
    return products.data
```

C. Connection Pooling:

```python
# In supabase_client.py
from supabase import create_client, Client
from contextlib import contextmanager

class SupabasePool:
    def __init__(self):
        self.client = create_client(SUPABASE_URL, SUPABASE_KEY)

    @contextmanager
    def get_client(self):
        yield self.client

pool = SupabasePool()

# Usage
with pool.get_client() as client:
    result = client.table("products").select("*").execute()
```

3. SECURITY ENHANCEMENTS

Current Security

```python
# ✅ Good: Password hashing
# ✅ Good: JWT tokens
# × Missing: Rate limiting on all endpoints
# × Missing: Input sanitization
# × Missing: SQL injection protection (Supabase handles this)
# × Missing: CORS restrictions (allow_origins=["*"])
```

Recommended Improvements

A. Environment Variables (.env):

```env
```

```
# Production - NEVER commit this!
SECRET_KEY=your_very_long_random_secret_key
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_KEY=your-supabase-service-role-key  # Use service role for backend!

# API Keys
DARAJA_CONSUMER_KEY=...
DARAJA_CONSUMER_SECRET=...
WHATSAPP_TOKEN=...
GROQ_API_KEY=...
```

B. CORS Configuration:

python

```python
# In main.py - Production
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://ayutech.co.ke",
        "https://admin.ayutech.co.ke",
        "http://localhost:3000",  # Development
    ],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
    allow_headers=["Authorization", "Content-Type"],
)
```

C. Input Validation:

python

```python
from pydantic import BaseModel, validator

class OrderCreate(BaseModel):
    customer_phone: str
    total_amount: float

    @validator('customer_phone')
    def validate_phone(cls, v):
        # Ensure Kenyan phone format
        if not v or len(v) < 9:
```

```
            raise ValueError('Invalid phone number')
        return v

    @validator('total_amount')
    def validate_amount(cls, v):
        if v <= 0:
            raise ValueError('Amount must be positive')
        return v
```

D. Audit Logging:

```
python

# Add audit log table
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID,
    action TEXT NOT NULL,
    resource TEXT NOT NULL,
    resource_id TEXT,
    details JSONB,
    ip_address TEXT,
    user_agent TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

# Log important actions
async def log_action(user_id, action, resource, resource_id, details):
    supabase.table("audit_logs").insert({
        "user_id": user_id,
        "action": action,
        "resource": resource,
        "resource_id": resource_id,
        "details": details
    }).execute()
```

4. MONITORING & OBSERVABILITY

What You Should Implement

A. Logging:

```python
import logging
from datetime import datetime

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('app.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)

# Use in endpoints
@router.post("/checkout")
async def checkout(payload: CheckoutRequest):
    logger.info(f"Checkout started for phone: {payload.phone}")
    try:
        result = await process_checkout(payload)
        logger.info(f"Checkout successful: {result['order_id']}")
        return result
    except Exception as e:
        logger.error(f"Checkout failed: {str(e)}")
        raise
```

B. Metrics (Prometheus + Grafana):

```python
from prometheus_client import Counter, Histogram, generate_latest

# Define metrics
orders_total = Counter('orders_total', 'Total orders placed')
orders_duration = Histogram('orders_duration_seconds', 'Order processing time')
errors_total = Counter('errors_total', 'Total errors', ['endpoint'])

@router.get("/metrics")
```

```
async def metrics():
    return Response(generate_latest(), media_type="text/plain")

@router.post("/checkout")
async def checkout():
    orders_total.inc()
    with orders_duration.time():
        return await process_checkout()
```

C. Health Checks:

```
python

@router.get("/health")
async def health_check():
    # Check database
    try:
        supabase.table("products").select("id").limit(1).execute()
        db_status = "healthy"
    except:
        db_status = "unhealthy"

    # Check external services
    return {
        "status": "healthy" if db_status == "healthy" else "unhealthy",
        "database": db_status,
        "timestamp": datetime.now().isoformat()
    }
```

5. SCALING STRATEGIES

When to Scale

Level 1: Single Server (0-1000 users/day)

- Run everything on one VPS
- Use Supabase for database
- No caching needed

Level 2: Split Backend + Frontend (1000-10,000 users/day)

- Backend on one server
- Frontend on another (CDN)
- Add Redis caching
- Use multiple workers (Gunicorn)

Level 3: Microservices (10,000+ users/day)

- Split into microservices
- Add message queue (RabbitMQ)
- Database read replicas
- Load balancer

6. CI/CD PIPELINE

What You Should Set Up

```yaml
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
```

```
      - name: Install dependencies
        run: |
          cd backend
          pip install -r requirements.txt

      - name: Run tests
        run: |
          cd backend
          pytest

      - name: Deploy to server
        uses: appleboy/ssh-action@v0.1.5
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SSH_PRIVATE_KEY }}
          script: |
            cd /var/www/ayutech
            git pull
            docker-compose down
            docker-compose up -d --build
```

📊 ARCHITECTURE PATTERNS TO KNOW

1. Repository Pattern

python

```python
# Instead of direct Supabase calls everywhere
class ProductRepository:
    @staticmethod
    def get_all():
        return supabase.table("products").select("*").execute()

    @staticmethod
    def get_by_category(category):
        return supabase.table("products").select("*").eq("category", category).execute
()

    @staticmethod
```

```
    def update_stock(product_id, quantity):
        return supabase.table("products").update({"stock_quantity": quantity}).eq("id",
product_id).execute()

# Usage
products = ProductRepository.get_all()
ProductRepository.update_stock("123", 5)
```

2. Service Pattern

```
python

# Business logic in services, not endpoints
class OrderService:
    def __init__(self):
        self.order_repo = OrderRepository()
        self.product_repo = ProductRepository()

    async def create_order(self, payload):
        # 1. Validate products
        # 2. Check stock
        # 3. Calculate total
        # 4. Create order
        # 5. Trigger payment
        return order
```

3. DTO Pattern (Data Transfer Object)

```
python

# Don't expose database models directly
class OrderDTO:
    def __init__(self, order_data):
        self.id = order_data.get("id")
        self.reference = order_data.get("order_reference")
        self.total = order_data.get("total_amount")
        self.status = order_data.get("status")
        self.created_at = order_data.get("created_at")

    def to_dict(self):
        return {
```

```
            "id": self.id,
            "reference": self.reference,
            "total": self.total,
            "status": self.status,
            "created_at": self.created_at
        }

# Usage
order_dto = OrderDTO(order)
return order_dto.to_dict()
```

4. Factory Pattern

```
python

class PaymentFactory:
    @staticmethod
    def create_payment(method, amount, phone):
        if method == "mpesa":
            return MpesaPayment(amount, phone)
        elif method == "cash":
            return CashPayment(amount)
        elif method == "bank":
            return BankTransferPayment(amount)
        else:
            raise ValueError("Unknown payment method")
```

🚀 NEXT STEPS CHECKLIST

Immediate (Week 1-2)

□ Set up environment variables properly

□ Add database indexes

□ Implement Redis caching

□ Add comprehensive logging

□ Set up health checks

Short-term (Month 1-2)

□ Write unit tests (pytest)

□ Add API documentation (Swagger/OpenAPI)

□ Implement CI/CD pipeline

□ Add monitoring (Prometheus/Grafana)

□ Set up error tracking (Sentry)

Medium-term (Month 3-4)

□ Migrate to microservices (if needed)

□ Add message queue (RabbitMQ)

□ Implement WebSocket for real-time updates

□ Add Elasticsearch for search

□ Set up database read replicas

Long-term (Month 6+)

□ Implement AI/ML for recommendations

□ Add predictive inventory management

□ Implement customer analytics dashboard

□ Add multi-tenant support

□ Internationalization (i18n)

💡 FINAL ADVICE

What You Should Know as a Software Engineer

1. **Always Think About Scale**
   - How many users will this handle?
   - What's the bottleneck?

- How will you monitor it?

2. **Security First**
   - Never trust user input
   - Always validate data
   - Use environment variables for secrets
   - Implement rate limiting
3. **Code Organization Matters**
   - Keep business logic separate from API
   - Use design patterns
   - Write tests
   - Document everything
4. **Learn the Ecosystem**
   - Docker for containerization
   - Kubernetes for orchestration
   - Cloud providers (AWS, GCP, Azure)
   - Infrastructure as Code (Terraform)
5. **Stay Curious**
   - Read other people's code
   - Follow best practices
   - Join communities (GitHub, Stack Overflow)
   - Keep learning!

🎉 You've Completed the Full Stack Journey!

You now know:

- ✅ How to build a full-stack application
- ✅ How to integrate payments (M-Pesa)
- ✅ How to use AI services

- ✅ How to handle WhatsApp notifications
- ✅ How to structure a production-ready application
- ✅ How to think about scaling and architecture

What would you like to learn next?

- 🐳 Docker & Containerization?
- ☁️ Cloud Deployment?
- 🔄 CI/CD Pipeline Setup?
- 📊 System Design Patterns?
- 🧪 Testing & Quality Assurance?

This response is AI-generated, for reference only.